

Logo
SUP'COM

Université de Carthage

Logo
organisme d'accueil

École Supérieure des Communications de Tunis (SUP'COM)

Cycle d'ingénieur : Génie des Réseaux et Systèmes Informatiques

RAPPORT DE PROJET DE FIN D'ÉTUDES

Présenté en vue de l'obtention du Diplôme National d'Ingénieur

DevOps Central Platform

Conception et mise en œuvre d'une chaîne DevSecOps
et GitOps de bout en bout sur cluster Kubernetes

Réalisé par

Abdelkader Ben Hmida

Encadrant académique

[Nom de l'encadrant académique]

Encadrant professionnel

[Nom de l'encadrant professionnel]

Membres du jury

Président : [Nom, grade]

Rapporteur : [Nom, grade]

Encadrant : [Nom de l'encadrant académique]

Année universitaire 2025 – 2026

Dédicaces

À mes parents, pour leur soutien constant et leur confiance sans réserve tout au long de mon parcours.

À ma famille, qui n'a jamais cessé de m'encourager.

À mes enseignants, qui m'ont transmis le goût de la rigueur et le sens de l'ingénierie.

À mes amis, compagnons de nuits blanches et de projets partagés.

Abdelkader

Remerciements

Je tiens à exprimer ma profonde gratitude à toutes les personnes qui ont contribué, de près ou de loin, à l'aboutissement de ce projet de fin d'études.

Mes remerciements s'adressent en premier lieu à mon encadrant académique, [Nom de l'encadrant académique], pour la qualité de son encadrement, la pertinence de ses remarques et la disponibilité dont il a fait preuve tout au long de ce travail.

J'exprime également ma reconnaissance à mon encadrant professionnel, [Nom de l'encadrant professionnel], pour la confiance qu'il m'a accordée, pour les échanges techniques qui ont orienté plusieurs décisions d'architecture de cette plateforme, et pour l'autonomie qu'il m'a laissée dans la conduite du projet.

Je remercie l'ensemble du corps enseignant de l'École Supérieure des Communications de Tunis (SUP'COM) pour la formation d'ingénieur solide qui m'a permis d'aborder ce sujet avec les fondamentaux nécessaires, en particulier en systèmes, en réseaux et en génie logiciel.

Enfin, j'adresse mes remerciements aux membres du jury qui me font l'honneur d'évaluer ce travail, ainsi qu'à ma famille et à mes camarades de promotion pour leur soutien tout au long de cette année.

Résumé

Ce projet de fin d'études porte sur la conception, la réalisation et la validation d'une plateforme DevOps complète, nommée *DevOps Central Platform*, couvrant l'intégralité du cycle de vie d'une application distribuée : du provisionnement de l'infrastructure jusqu'à la supervision en production.

La plateforme repose sur une infrastructure virtualisée libvirt/KVM décrite entièrement en Infrastructure as Code avec Terraform, configurée de manière idempotente par Ansible, et hébergeant un cluster Kubernetes 1.28 multi-nœuds installé par kubeadm avec le plugin réseau Calico. La charge applicative est constituée de trois microservices Python FastAPI conteneurisés selon un modèle multi-stage durci, déployés via Kustomize sur trois environnements distincts (dev, staging, production).

La sécurité est intégrée à chaque étape de la chaîne plutôt qu'ajoutée a posteriori : détection de secrets par Gitleaks, analyse de vulnérabilités des images par Trivy, audit des dépendances par pip-audit, politiques d'admission conftest et Kyverno, NetworkPolicies en *default-deny*, Pod Security Admission en mode *restricted*, et gestion dynamique des secrets par HashiCorp Vault selon un contrat *fail-closed*. L'automatisation du déploiement suit un modèle hybride : une chaîne d'intégration continue GitHub Actions en mode *push* pour la construction et la validation, et un moteur GitOps ArgoCD en mode *pull* pour la réconciliation continue entre l'état déclaré dans Git et l'état réel du cluster.

L'observabilité est assurée par une double chaîne : métriques (Prometheus, AlertManager, Grafana) et logs centralisés (Elasticsearch, Filebeat, Logstash, Kibana), avec des objectifs de niveau de service (SLO) formalisés en règles d'alerte exécutables. Sept scénarios de validation de bout en bout, incluant des scénarios d'incident volontairement provoqués, démontrent le comportement réel de la plateforme en condition nominale comme en condition dégradée.

Mots clés : DevOps, DevSecOps, GitOps, Kubernetes, Terraform, Ansible, Infrastructure as Code, conteneurisation, intégration continue, observabilité, SLO, gestion des secrets.

Abstract

This graduation project covers the design, implementation and validation of a complete DevOps platform, named *DevOps Central Platform*, spanning the full lifecycle of a distributed application, from infrastructure provisioning to production monitoring.

The platform is built on a libvirt/KVM virtualized infrastructure fully described as Infrastructure as Code with Terraform, configured idempotently with Ansible, and hosting a multi-node Kubernetes 1.28 cluster installed with kubeadm and the Calico network plugin. The workload consists of three containerized Python FastAPI microservices built from a hardened multi-stage image model and deployed through Kustomize across three environments (dev, staging, production).

Security is embedded at every stage of the chain rather than added afterwards: secret detection with Gitleaks, image vulnerability scanning with Trivy, dependency auditing with pip-audit, admission policies with conftest and Kyverno, default-deny NetworkPolicies, restricted Pod Security Admission, and dynamic secret management with HashiCorp Vault under a fail-closed contract. Deployment automation follows a hybrid model: a GitHub Actions push-based continuous integration chain for build and validation, and an ArgoCD pull-based GitOps engine for continuous reconciliation between the state declared in Git and the actual cluster state.

Observability relies on two complementary chains: metrics (Prometheus, AlertManager, Grafana) and centralized logs (Elasticsearch, Filebeat, Logstash, Kibana), with service level objectives formalized as executable alerting rules. Seven end-to-end validation scenarios, including deliberately triggered incident scenarios, demonstrate the actual behaviour of the platform under both nominal and degraded conditions.

Keywords: DevOps, DevSecOps, GitOps, Kubernetes, Terraform, Ansible, Infrastructure as Code, containerization, continuous integration, observability, SLO, secret management.

Table des matières

Dédicaces	i
Remerciements	iii
Résumé	v
Liste des figures	xii
Liste des tableaux	xiii
Liste des acronymes	xv
Introduction générale	1
I Architecture et choix de conception	5
1 Vue d'ensemble de la plateforme	7
1.1 Ce que la plateforme doit prouver	7
1.2 Les six couches et leur enchaînement	8
1.3 Topologie réseau et dimensionnement	9
2 Infrastructure as Code : comment une machine virtuelle naît	11
2.1 L'hyperviseur : donner un corps à la machine	11
2.2 Terraform : déclarer l'état voulu	12
2.3 cloud-init : durcir avant même le premier login	13
2.4 Ansible : de l'Ubuntu générique au nœud Kubernetes	14
3 Orchestration : d'un commit à un pod en production	17
3.1 Construire : Docker et le modèle multi-stage	17
3.2 Exécuter : containerd et la fin de Docker en production	18
3.3 Orchestrer : Kubernetes 1.28 et Calico	18
3.4 Personnaliser par environnement : Kustomize	19
4 Applications et gestion dynamique des secrets	21
4.1 Trois services, une seule plateforme	21
4.2 Le contrat fail-closed	21
4.3 La bibliothèque partagée : un seul comportement pour trois services . .	23
4.4 Vault : distribuer les secrets sans les figer dans Git	24
5 Observabilité et objectifs de niveau de service	27

5.1	Prometheus : la collecte de métriques	27
5.2	AlertManager : du seuil PromQL à l'alerte routée	28
5.3	Grafana : les tableaux de bord comme du code	29
5.4	ELK : quand les métriques ne suffisent plus	30
5.5	Le modèle de menaces vu à travers l'observabilité	30
5.6	SLO : transformer la supervision en contrat	31
6	Intégration et déploiement continu	33
6.1	GitHub Actions : vérifier avant de publier	33
6.2	ArgoCD : le cluster tire son état depuis Git	34
6.3	Les contrôles DevSecOps intégrés au flux	36
II	Analyse critique	37
7	Choix techniques : synthèse comparative	39
7.1	Trois familles d'alternatives récurrentes	39
7.2	Le fil directeur de ces arbitrages	40
8	Limites assumées et dette technique	41
8.1	Dette de sécurité assumée	41
8.2	Dette d'exécution	41
8.3	Dette cosmétique et résiduelle	42
9	Validation : scénarios de bout en bout	43
9.1	Scénario 1 — Du commit au déploiement	43
9.2	Scénario 2 — Incident Vault indisponible	43
9.3	Scénario 3 — Fuite mémoire et OOMKill	44
9.4	Scénario 4 — Stress HPA et anti-flapping	44
9.5	Scénario 5 — Rollback GitOps après régression	44
9.6	Scénario 6 — Rotation complète des secrets	45
9.7	Scénario 7 — Détection de drift supply-chain	45
9.8	Outils de vérification continue	45
	Conclusion générale et perspectives	47
	Bibliographie	49
A	Commandes utiles	51
B	Variables d'environnement	53
C	Index des captures demandées	55
D	Arborescence commentée du dépôt	59
E	Glossaire	61
F	Journal des décisions (extraits ADR)	63

G	Cartographie des ports et endpoints	65
H	Checklists d'exploitation	67
	H.1 Quotidienne (5 min)	67
	H.2 Hebdomadaire (20 min)	67
	H.3 Mensuelle (1 h)	67
I	Banque de questions / réponses éclair	69
J	Matrice outils × dimensions DevOps	73
K	Feuille de route de durcissement	75
	K.1 Vault : du dev-mode à la production	75
	K.2 Politiques : Audit → Enforce	75
	K.3 CI/CD	75
	K.4 Application	75
L	Correspondance alertes / incidents / runbooks	77
M	Guide de réalisation des captures	79
	M.1 Conventions générales	79
	M.2 Ordre conseillé de prise de captures	79
	M.3 Rappels utiles pendant la capture	80

Liste des figures

1.1	Schéma d'architecture général redessiné ou photographié depuis un tableau...	9
1.2	Sortie de <code>virsh net-dumpxml devops-platform-net</code> ou vue network de...	10
2.1	<code>Virsh list --all</code> montrant les 3 domaines en running + <code>virsh dumpxml master-01</code> ...	11
2.2	Terraform plan : create des domaines/volumes/cloudinit disks avec le résumé...	13
2.3	Tentative <code>ssh root@192.168.56.10</code> affichant Permission denied, puis connexion...	14
2.4	Fin de run vert : PLAY RECAP rc=0, changed/unreachable, temps total (callback...	16
2.5	<code>Kubectl get nodes -o wide</code> juste après le run : 3 nœuds Ready v1.28.x	16
3.1	Test CNI : DNS + ping entre deux Pods de nœuds différents (<code>kubectl exec +</code> ...	18
3.2	Preuve default-deny : un Pod sans label ne peut joindre users-service, un Pod...	20
4.1	Pod en crashloop après suppression du secret : <code>kubectl get pods + logs</code> ...	22
4.2	<code>Kubectl get pods -n vault + vault status exec</code> dans le pod : initialized true...	25
5.1	Dashboard Error Rate SLO avec ligne de seuil 1 % visible	29
6.1	Job trivy-scan : table CRITICAL/HIGH vide + upload SARIF réussi	34
6.2	UI ArgoCD : tuiles des 12 Applications toutes Synced/Healthy	35
6.3	Démonstration selfHeal : <code>kubectl scale deploy users-service --replicas=5</code> ...	36
6.4	<code>Kustomize build k8s/apps/base conftest test --policy k8s/policies/conftest</code> ...	36
9.1	<code>Scripts/validate-platform.sh</code> complet : les 7 contrôles + bonus en vert avec...	46
H.1	Checklist hebdo exécutée en terminal : <code>validate-platform 7+bonus</code> verts	68

M.1 Capture « carte d'identité » de la plateforme : kubectl top nodes + kubectl...	80
---	----

Liste des tableaux

1.1	Namespaces et profils de sécurité	9
1.3	Topologie par défaut (<code>worker_count</code> paramétrable de 1 à 32)	10
3.1	HPA — comportement anti-flapping	19
3.2	Trois postures pour trois environnements	20
5.1	Catalogue complet des alertes	28
5.3	Les quatre dashboards provisionnés	29
5.5	Lecture croisée menace / contre-mesure observée	30
7.1	Alternatives structurantes écartées et raison commune	39
B.1	Variables d’environnement	53
C.1	Index des captures demandées	55
F.1	Journal des décisions (extraits ADR)	63
G.1	Cartographie des ports et endpoints	65
I.1	Banque de questions / réponses éclair	69
J.1	Matrice outils dimensions DevOps	73
L.1	Correspondance alertes / incidents / runbooks	77

Liste des acronymes

Acronyme	Signification
API	<i>Application Programming Interface</i>
CD	<i>Continuous Deployment</i> : déploiement continu
CI	<i>Continuous Integration</i> : intégration continue
CNI	<i>Container Network Interface</i>
CRD	<i>Custom Resource Definition</i>
CRI	<i>Container Runtime Interface</i>
CVE	<i>Common Vulnerabilities and Exposures</i>
DNS	<i>Domain Name System</i>
ELK	<i>Elasticsearch, Logstash, Kibana</i>
GHCR	<i>GitHub Container Registry</i>
HCL	<i>HashiCorp Configuration Language</i>
HPA	<i>Horizontal Pod Autoscaler</i>
IaC	<i>Infrastructure as Code</i>
JWT	<i>JSON Web Token</i>
KVM	<i>Kernel-based Virtual Machine</i>
KSM	<i>kube-state-metrics</i>
NAT	<i>Network Address Translation</i>
OCI	<i>Open Container Initiative</i>
OIDC	<i>OpenID Connect</i>
OOM	<i>Out Of Memory</i>
PDB	<i>Pod Disruption Budget</i>
PSA	<i>Pod Security Admission</i>
RBAC	<i>Role-Based Access Control</i>
SLA	<i>Service Level Agreement</i>
SLI	<i>Service Level Indicator</i>
SLO	<i>Service Level Objective</i>
SSH	<i>Secure Shell</i>
TLS	<i>Transport Layer Security</i>
VM	<i>Virtual Machine</i> : machine virtuelle
YAML	<i>YAML Ain't Markup Language</i>

Introduction générale

Contexte

L'industrie logicielle a profondément modifié, au cours de la dernière décennie, la manière dont une application passe du poste du développeur à l'environnement de production. Le modèle historique, où une équipe de développement livrait un artefact à une équipe d'exploitation distincte, a montré ses limites : délais de mise en production longs, environnements divergents, incidents difficiles à reproduire et responsabilités diluées. Le mouvement DevOps est né de ce constat. Il ne se réduit ni à un outil ni à un poste : il désigne un ensemble de pratiques d'ingénierie visant à raccourcir le cycle entre une modification de code et sa mise à disposition fiable aux utilisateurs, en automatisant systématiquement ce qui peut l'être et en rendant observable ce qui ne peut pas l'être.

Deux évolutions ont accompagné ce mouvement. La première est l'intégration de la sécurité au sein même de la chaîne de livraison (le DevSecOps) qui déplace les contrôles de sécurité vers l'amont du cycle plutôt que de les concentrer dans un audit final. La seconde est l'adoption du modèle GitOps, qui fait du dépôt Git la source unique de vérité de l'état désiré de l'infrastructure, un agent installé dans le cluster se chargeant de réconcilier en continu l'état réel avec cet état déclaré.

Problématique

La question à laquelle ce travail répond peut se formuler simplement : *comment une équipe d'ingénierie fait-elle fonctionner une application en production de manière reproductible, fiable, sécurisée et observable, sans intervention manuelle ?*

Cette question, apparemment unique, en recouvre en réalité cinq :

- comment garantir que l'infrastructure soit **reproductible** à l'identique, et non le résultat d'une suite d'actions manuelles non documentées ?
- comment rendre la plateforme **fiable**, c'est-à-dire capable de détecter ses propres défaillances et de s'en remettre sans opérateur ?
- comment intégrer la **sécurité** au flux de livraison de façon à ce qu'elle bloque effectivement les régressions, plutôt que de produire des rapports lus après coup ?

- comment rendre le système **observable**, au sens où l'on peut répondre à une question non anticipée sur son comportement sans redéployer ?
- comment supprimer l'**intervention manuelle** du chemin de déploiement, tout en conservant un point de contrôle humain sur la production ?

Objectifs du projet

L'objectif de ce projet de fin d'études est de construire une plateforme complète apportant une réponse concrète et mesurable à chacune de ces cinq questions, puis d'en démontrer le fonctionnement par des scénarios exécutables. Les objectifs opérationnels retenus sont les suivants :

1. décrire l'intégralité de l'infrastructure en code versionné, de la machine virtuelle jusqu'aux objets Kubernetes ;
2. automatiser la configuration des nœuds et l'installation du cluster de manière idempotente et rejouable ;
3. conteneuriser une charge applicative représentative selon les bonnes pratiques de sécurité des images ;
4. intégrer une chaîne de contrôles de sécurité bloquante dans le pipeline d'intégration continue ;
5. mettre en œuvre un déploiement GitOps avec réconciliation continue et possibilité de retour arrière par simple opération Git ;
6. instrumenter la plateforme avec une double chaîne d'observabilité, métriques et logs, et formaliser des objectifs de niveau de service vérifiables ;
7. valider l'ensemble par des scénarios de bout en bout, incluant des scénarios de défaillance provoquée.

Méthodologie de travail

Le projet a été conduit de manière itérative et incrémentale : chaque brique a été construite, éprouvée, puis intégrée à la précédente avant de passer à la suivante, selon l'ordre naturel de la chaîne de livraison (infrastructure, configuration, conteneurisation, orchestration, intégration continue, déploiement, observabilité).

Un parti pris méthodologique structure l'ensemble du travail : **documenter les échecs autant que les succès**. Plusieurs décisions techniques présentées dans ce rapport ne sont pas issues d'une lecture de documentation mais d'incidents réellement rencontrés pendant la réalisation : saturation mémoire des nœuds, terminaisons pour dépassement

de mémoire, autoscaler bloqué à sa borne supérieure. Chaque règle d'alerte de la plateforme porte à ce titre une étiquette renvoyant au post-mortem correspondant. Cette démarche est conforme à la pratique d'ingénierie de fiabilité, où l'incident constitue une source d'apprentissage documentée et non un événement que l'on efface.

Enfin, l'ensemble du contenu technique de ce rapport a été établi à partir de la lecture directe du code source effectivement actif dans le dépôt : les valeurs, les versions et les configurations citées sont celles réellement en vigueur, et non des valeurs par défaut recopiées d'une documentation externe.

Organisation du rapport

Ce rapport s'organise en neuf chapitres, regroupés en deux parties : une partie architecture qui suit le **flux réel de la plateforme**, de la naissance d'une machine virtuelle jusqu'à l'alerte qui signale un incident, et une partie analyse critique qui prend du recul sur ces choix.

Le **chapitre 1** pose la vue d'ensemble : les six couches de la plateforme, la topologie réseau et le dimensionnement retenu. Le **chapitre 2** raconte comment une machine virtuelle naît et se configure : libvirt/KVM, cloud-init, Terraform et Ansible y sont traités comme les étapes successives d'un seul et même processus de provisionnement, non comme quatre fiches indépendantes. Le **chapitre 3** suit le chemin inverse, celui d'un déploiement applicatif : construction des images Docker, exécution par containerd, orchestration Kubernetes et personnalisation Kustomize par environnement.

Le **chapitre 4** présente l'architecture applicative et la gestion des secrets : les trois microservices FastAPI étant quasi identiques, l'accent est mis sur ce qui les distingue réellement et sur le contrat fail-closed avec HashiCorp Vault, plutôt que sur trois descriptions redondantes. Le **chapitre 5** regroupe en un seul chapitre cohérent l'ensemble de la chaîne d'observabilité : métriques Prometheus, alerting, tableaux de bord Grafana, logs centralisés ELK et objectifs de niveau de service. Le **chapitre 6** suit le pipeline GitHub Actions et le déploiement continu ArgoCD comme un seul flux, du commit au cluster.

Le **chapitre 7** ouvre la partie analyse critique en confrontant les choix techniques retenus aux alternatives sérieusement envisagées, sous forme transversale plutôt qu'outil par outil. Le **chapitre 8** assume les limites et la dette technique du projet telles qu'elles existent réellement dans le code. Le **chapitre 9** rejoue sept scénarios de validation de bout en bout, dont plusieurs scénarios d'incident volontairement provoqués, qui donnent la preuve exécutable de ce que les chapitres précédents affirment.

Une conclusion générale, une bibliographie et un ensemble d'annexes techniques complètent le document.

Partie I

Architecture et choix de conception

CHAPITRE 1

Vue d'ensemble de la plateforme

1.1 Ce que la plateforme doit prouver

DevOps Central Platform est une plateforme d'infrastructure DevOps de bout en bout, construite autour de trois microservices **FastAPI** (*Users, Products, Orders*). L'application elle-même est volontairement simple : ce que ce rapport documente n'est pas son code métier, mais la chaîne qui l'amène en production et la maintient là de façon reproductible, fiable, sécurisée et observable, sans intervention manuelle.

Le projet tourne sur un **homelab libvirt/KVM** : aucun service cloud facturé n'est utilisé, la plateforme entière est reproductible sur une station Linux personnelle, tout en restant structurellement proche d'une production réelle (VMs, SSH, systemd, cluster Kubernetes multi-nœuds). Ce choix n'est pas anecdotique : il impose des contraintes de dimensionnement réelles (mémoire, disque) qui reviennent tout au long de ce rapport et qui ont, à plusieurs reprises, dicté des décisions d'architecture.

Chaque chapitre qui suit répond à un fragment de la question fondatrice du projet : *comment une équipe d'ingénierie fait-elle tourner une application en production de façon reproductible, fiable, sécurisée et observable, sans intervention manuelle ?*

- *reproductible* → chapitre 2 (Terraform, cloud-init, Ansible) ;
- *fiable* → chapitre 3 (Kubernetes, probes, HPA, PDB) ;
- *sécurisée* → chapitres 4 et 6 (Vault, scans CI, admission) ;
- *observable* → chapitre 5 (Prometheus, AlertManager, Grafana, ELK) ;
- *sans intervention manuelle* → chapitre 6 (GitHub Actions, ArgoCD).

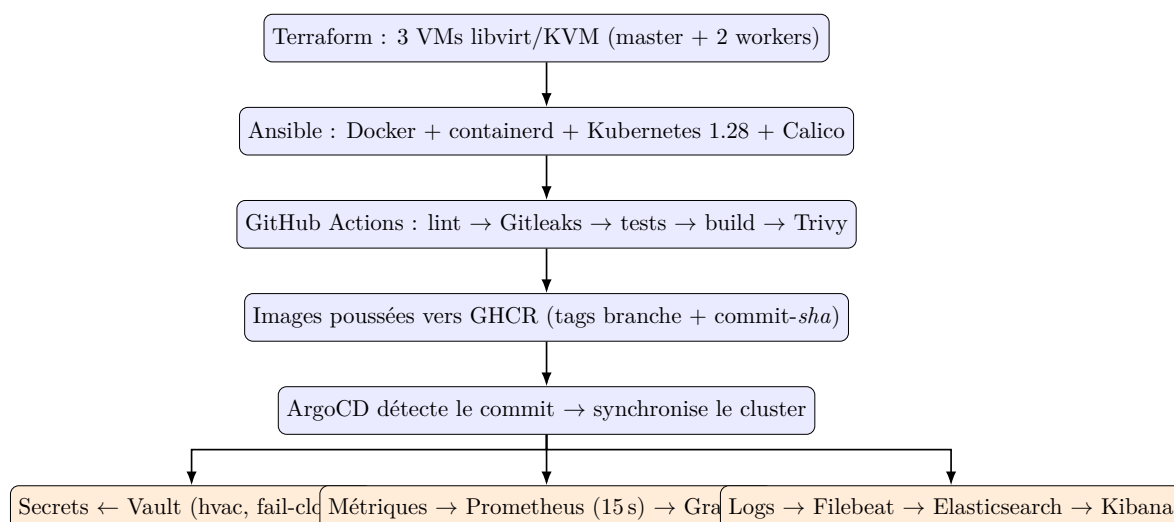
Philosophie : documenter aussi les échecs

Le projet ne se limite pas à faire fonctionner l'architecture : il documente comment elle échoue en pratique et comment ces échecs se corrigent. Les alertes plateforme (OOMKill, HPA bloqué au maximum, saturation mémoire) correspondent à des incidents réellement rencontrés pendant le développement, repris comme scénarios de validation au chapitre 9.

1.2 Les six couches et leur enchaînement

Plutôt que de traiter chaque outil comme une entrée isolée d'un catalogue, ce rapport suit le flux réel par lequel une modification de code devient un comportement observable en production. Ce flux traverse six couches, chacune dépendant strictement de celle qui la précède :

1. **Infrastructure as Code** : Terraform crée trois machines virtuelles reproductibles (chapitre 2) ;
2. **Configuration automatisée** : cloud-init puis Ansible installent et durcissent ces machines jusqu'à obtenir un cluster Kubernetes (chapitre 2) ;
3. **Conteneurisation et orchestration** : Docker empaquette, containerd exécute, Kubernetes et Kustomize orchestrent et personnalisent par environnement (chapitre 3) ;
4. **Applications et secrets** : les microservices FastAPI consomment des secrets dynamiques distribués par Vault selon un contrat fail-closed (chapitre 4) ;
5. **Sécurité et livraison continue** : GitHub Actions vérifie, scanne et publie ; ArgoCD réconcilie en continu l'état du cluster avec Git (chapitre 6) ;
6. **Observabilité** : Prometheus, AlertManager, Grafana et la pile ELK rendent visible ce qui se passe réellement, et des SLO chiffrés le rendent contractuel (chapitre 5).



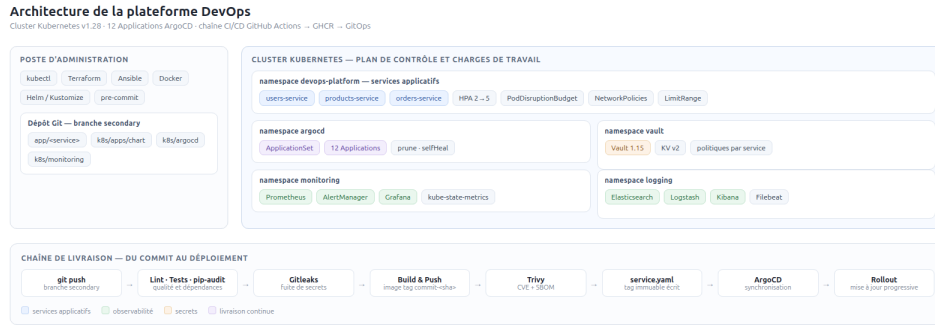


Figure 1.1. Schéma d'architecture général redessiné ou photographié depuis un tableau blanc / outil de diagramme : les 3 VMs, le cluster K8s, les namespaces (devops-platform, monitoring, vault, argocd), les flux CI et GitOps.

Cette organisation en couches se retrouve dans les namespaces du cluster : chacun porte un profil de sécurité Pod Security Admission (PSA) proportionné à son rôle.

Tableau 1.1. Namespaces et profils de sécurité

Namespace	PSA	Contenu
devops-platform	restricted	3 services (SA dédiés, HPA, PDB, netpol), quotas, Secret vault-root-token
monitoring	restricted	Prometheus, AlertManager, Grafana+sidecar, ES, Filebeat DS, Logstash, Kibana, KSM, ServiceMonitors, rules SLO, 4 dashboards CM
vault	baseline	Vault 1.15.2 dev-mode + Job setup (IPC_LOCK requis)
argocd	—	ArgoCD v3.5.1 complet + Applications
kube-system calico-system	/ —	composants cluster (hors périmètre applicatif)

1.3 Topologie réseau et dimensionnement

Terraform crée un réseau libvirt dédié, **devops-platform-net**, en mode isolé avec NAT vers l'extérieur, domaine DNS local **devops.local**, sous-réseau **192.168.56.0/24** validé (plage RFC1918 obligatoire), relais DNS vers **1.1.1.1** et **8.8.8.8**. Les nœuds reçoivent des adresses statiques hors de la plage DHCP (**192.168.56.100–200**) :

Trois plans d'adressage coexistent sans conflit : réseau des VMs (**192.168.56.0/24**), CIDR pods (**192.168.0.0/16**), CIDR services (**10.96.0.0/12**) — séparation classique exigée par kubeadm.

Nœud	IP statique	Rôle cluster	Dimensionnement
master-01	192.168.56.10	control-plane	2 vCPU / 4 Go / 20 Go qcow2
worker-01	192.168.56.11	worker	2 vCPU / 4 Go / 20 Go qcow2
worker-02	192.168.56.12	worker	2 vCPU / 4 Go / 20 Go qcow2

Tableau 1.3. Topologie par défaut (`worker_count` paramétrable de 1 à 32)

Un dimensionnement mémoire justifié par incident

La variable Terraform `vm_memory_mb` est fixée à **4096**, avec un commentaire explicite dans le code : à 2048 Mo, la pile complète (ELK + ArgoCD + supervision) fait basculer le kubelet en `NodeStatusUnknown` sous pression mémoire. Ce type de décision justifiée par un incident réellement rencontré, plutôt que recopiée d'une documentation, traverse tout ce rapport.

```

devops@homelab:~/rapport$ virsh net-list --all; echo; virsh net-dumpxml devops-platform-net
Name                               State    Autostart Persistent
-----
default                            active   yes         yes
devops-platform-net               active   no          yes

<network connections='3'>
  <name>devops-platform-net</name>
  <uuid>cb1fa4af-652e-44bd-abdc-5d5fefb66044</uuid>
  <forward mode='nat'>
    <nat>
      <port start='1024' end='65535' />
    </nat>
  </forward>
  <bridge name='virbr1' stp='on' delay='0' />
  <mac address='52:54:00:c3:f9:90' />
  <domain name='devops.local' localOnly='yes' />
  <dns enable='yes'>
    <forwarder addr='1.1.1.1' />
    <forwarder addr='8.8.8.8' />
  </dns>
  <ip address='192.168.56.1' netmask='255.255.255.0'>
    <dhcp>
      <range start='192.168.56.100' end='192.168.56.200' />
    </dhcp>
  </ip>
</network>

```

Figure 1.2. Sortie de `virsh net-dumpxml devops-platform-net` ou vue network de virt-manager montrant le réseau NAT, la plage DHCP et les interfaces des 3 VMs.

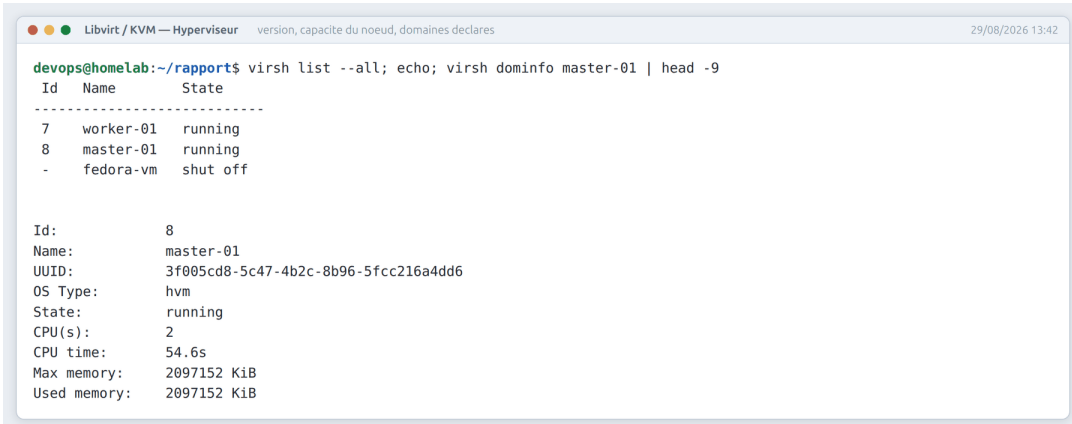
CHAPITRE 2

Infrastructure as Code : comment une machine virtuelle naît

Ce chapitre suit une seule question du début à la fin : *comment un terraform apply devient-il, quelques minutes plus tard, un nœud Kubernetes prêt à recevoir du trafic ?* Quatre outils interviennent successivement dans cette naissance — libvirt/KVM, cloud-init, Terraform, Ansible — et sont ici présentés comme les étapes d'un seul processus plutôt que comme quatre fiches indépendantes.

2.1 L'hyperviseur : donner un corps à la machine

KVM (Kernel-based Virtual Machine) est l'hyperviseur intégré au noyau Linux ; libvirt en est l'API de gestion standard. Le choix retenu privilégie de *vraies* VM Linux — kernel réel, systemd, SSH — plutôt qu'un cluster en conteneur unique (kind, minikube) qui masquerait la mécanique réseau et systemd, ou qu'un cloud public qui introduirait une facturation permanente contraire au principe homelab. Chaque nœud est une machine **q35** (chipset PCI-E moderne), disque **qcow2** cloné en copie-sur-écriture depuis une image Ubuntu Server 22.04, carte réseau **virtio**, console série et VNC restreinte à 127.0.0.1.



```
devops@homelab:~/rapports$ virsh list --all; echo; virsh dominfo master-01 | head -9
```

Id	Name	State
7	worker-01	running
8	master-01	running
-	fedora-vm	shut off


```
Id: 8
Name: master-01
UUID: 3f005cd8-5c47-4b2c-8b96-5fcc216a4dd6
OS Type: hvm
State: running
CPU(s): 2
CPU time: 54.6s
Max memory: 2097152 KiB
Used memory: 2097152 KiB
```

Figure 2.1. `virsh list --all` montrant les 3 domaines en running + `virsh dumpxml master-01` extraits memory/vcpu.

2.2 Terraform : déclarer l'état voulu

Terraform pilote libvirt de façon déclarative : le fichier d'état permet de détecter la dérive, de prévisualiser (`plan`) et de détruire proprement. Les versions sont volontairement épinglées à une borne haute plutôt qu'à l'actualité :

```
terraform {
  required_version = "~> 1.5" # borne haute : éviter ruptures 1.6+
  required_providers {
    libvirt = { source = "dmacvicar/libvirt", version = "~> 0.9" } # 0.9.8
    local = { source = "hashicorp/local", version = "~> 2.5" } # 2.9.0
  }
}
```

Listing 2.1. `main.tf` : épinglage des versions (extrait)

Sept ressources composent l'infrastructure : le réseau NAT dédié, un volume `qcow2` de base, un volume par nœud (`for_each`), un disque `cloud-init` par nœud, la VM elle-même, un garde-fou (`terraform_data.ssh_key_guard`, qui fait échouer l'apply immédiatement si aucune clé SSH publique n'est trouvée) et surtout le rendu de l'inventaire Ansible :

```
terraform refresh -input=false
terraform apply -input=false -auto-approve -target=local_file.ansible_inventory
cp terraform/inventory.generated.ini ansible/inventory.ini
```

Listing 2.2. Terraform, seule source de vérité de l'inventaire (`scripts/generate-inventory.sh`)

Faire dériver l'inventaire Ansible directement de l'état Terraform supprime une classe entière d'erreurs : il ne peut jamais diverger de l'infrastructure réelle. Le backend reste local (`terraform.tfstate`, gitignored) pour ce lab, mais le contrat de production est déjà écrit en commentaire dans `backend.tf` : backend S3, verrous DynamoDB, chiffrement — motivé par l'incident classique des appliques concurrents corrompant un état non verrouillé.

```

devops@homelab:~/rapports$ cd terraform && terraform plan -no-color -input=false 2>&1 | tail -20
# terraform_data.ssh_key_guard will be created
+ resource "terraform_data" "ssh_key_guard" {
+   id       = (known after apply)
+   input    = (sensitive value)
+   output   = (known after apply)
+ }

Plan: 16 to add, 0 to change, 0 to destroy.

Changes to Outputs:
+ ansible_inventory_content = (sensitive value)
+ ansible_inventory_file    = (sensitive value)
+ master_ip                 = (sensitive value)
+ node_ips                  = (sensitive value)
+ worker_ips                = (sensitive value)

Note: You didn't use the -out option to save this plan, so Terraform can't
guarantee to take exactly these actions if you run "terraform apply" now.

```

Figure 2.2. terraform plan : create des domaines/volumes/cloudinit disks avec le résumé final « Plan: N to add, 0 to change, 0 to destroy ».

2.3 cloud-init : durcir avant même le premier login

Chaque VM reçoit un disque cloud-init généré depuis le template `cloud-init.tpl`, appliqué au tout premier boot, *avant* qu'Ansible n'intervienne :

```

ssh_pwauth: false
disable_root: true
users:
  - name: devops
    sudo: "ALL=(ALL) NOPASSWD:ALL" # fenetre de provisionnement, voir texte
    lock_passwd: false # cle SSH uniquement, aucun mot de passe
runcmd:
  - sed -i 's/^#\?PermitRootLogin.*/PermitRootLogin no/' /etc/ssh/sshd_config
  - sed -i 's/^#\?PasswordAuthentication.*/PasswordAuthentication no/' /etc/ssh/
    sshd_config

```

Listing 2.3. Template cloud-init : durcissement appliqué avant toute autre étape

Le compte `devops` n'a **aucun mot de passe** (clé SSH uniquement) ; sudo est *temporairement* large (`NOPASSWD:ALL`), le temps que les rôles Ansible installent des paquets et écrivent dans `/etc` — une règle restreinte dès le boot ferait échouer la toute première tâche du playbook. Ce sudo large sera resserré plus tard par un rôle Ansible dédié (§2.4). `fail2ban` est installé d'emblée, et la partition racine s'étend automatiquement (`growpart`). Appliquer ce durcissement au premier boot garantit qu'aucune VM ne passe un seul instant dans un état faible.



```
cloud-init — Durcissement SSH  mot de passe desactive, connexion par cle  29/08/2026 13:42

devops@homelab:~/rapport$ echo '# connexion root : refusee (disable_root)' ;
timeout 8 ssh -o StrictHostKeyChecking=no -o BatchMode=yes root@192.168.56.10 true 2>&1 | tail -1 ;
echo; echo '# connexion devops par cle : acceptee' ;
timeout 8 ssh -o StrictHostKeyChecking=no -o BatchMode=yes devops@192.168.56.10 'id -un ;
grep -E "^PasswordAuthentication|^PermitRootLogin" /etc/ssh/sshd_config.d/*.conf /etc/ssh/sshd_config 2>/dev/null | head
-3' 2>/dev/null
# connexion root : refusee (disable_root)
root@192.168.56.10: Permission denied (publickey).

# connexion devops par cle : acceptee
devops
/etc/ssh/sshd_config.d/60-cloudimg-settings.conf:PasswordAuthentication no
/etc/ssh/sshd_config:PermitRootLogin no
/etc/ssh/sshd_config:PasswordAuthentication no
```

Figure 2.3. Tentative `ssh root@192.168.56.10` affichant `Permission denied`, puis connexion `devops` réussie par clé.

2.4 Ansible : de l'Ubuntu générique au nœud Kubernetes

Ansible automatise la configuration par SSH sans agent, de façon idempotente. Le playbook enchaîne sept plays dans un ordre strict :

```
- hosts: all # attend cloud-init avant de toucher a apt
- hosts: all, roles: [docker], tags: docker
- hosts: all, roles: [k8s_common], tags: k8s
- hosts: masters, roles: [k8s_master], tags: master
- hosts: workers, roles: [k8s_worker], serial: 1, tags: worker
- hosts: all, roles: [k8s_reset], when: reset_confirmed|default(false)|bool, tags:
  [reset, never]
- hosts: all, roles: [harden_sudo], tags: [harden, never] # non joue par default
```

Listing 2.4. `playbook.yml` : structure globale

Les valeurs `cluster` (`k8s_version: "1.28"`, `calico_version: "v3.26.1"`, `pod_cidr`, `service_cidr`) sont centralisées dans `group_vars/all.yml` et partagées par tous les rôles.

docker installe Docker CE et `containerd` depuis le dépôt officiel, puis pose un réglage discret mais critique : la génération de `/etc/containerd/config.toml` suivie d'une bascule vers le pilote de `cgroups systemd`.

```
- name: Set SystemdCgroup = true
  lineinfile:
    path: /etc/containerd/config.toml
    regexp: '^s+SystemdCgroup\s*='
    line: ' SystemdCgroup = true'
  notify: Restart containerd
```

Listing 2.5. Bascule cruciale : pilote `cgroup systemd` pour `containerd`

Ce même rôle prépare aussi le noyau pour Kubernetes : swap désactivé (le kubelet refuse de fonctionner avec), modules `overlay` et `br_netfilter` chargés et persistés, sysctls de forwarding IPv4 posés. `kubelet`, `kubeadm` et `kubectl` sont installés **épinglés** sur 1.28.* puis verrouillés par `apt-mark hold` : double verrou (version exacte + hold) qui garantit qu'aucun `apt upgrade` ne fera jamais dériver la version du cluster.

k8s_master initialise le control-plane avec des paramètres entièrement explicites (adresse d'annonce, CIDR pods/services, socket CRI) :

```
kubeadm init --apiserver-advertise-address={{ ansible_host }} \
  --pod-network-cidr={{ pod_cidr }} --service-cidr={{ service_cidr }} \
  --cri-socket={{ cri_endpoint }} --upload-certs \
  --skip-phases=addon/coredns,addon/kube-proxy
```

Listing 2.6. kubeadm init : addons différés pour éviter les CrashLoop initiaux

Les phases addon (CoreDNS, kube-proxy) sont volontairement différées : si CoreDNS démarre avant qu'un CNI ne fournisse le réseau des pods, il boucle en CrashLoop. Calico v3.26.1 est ensuite installé via l'opérateur Tigera, avec une garde stricte : l'opérateur doit être **Available**, puis **tous** les nœuds doivent passer **Ready**, sinon le run échoue net (« un cluster demi-configuré refuse de se déclarer prêt »). Le join command est généré à la volée, marqué `no_log: true` pour ne jamais apparaître dans les logs, et stocké en fichiers 0600/0700.

k8s_worker lit ce join command sur le master et rejoint via le module `command` : (et non `shell` : , pour éviter toute injection), avec jointure `serial: 1` : progressive, pour détecter rapidement un problème.

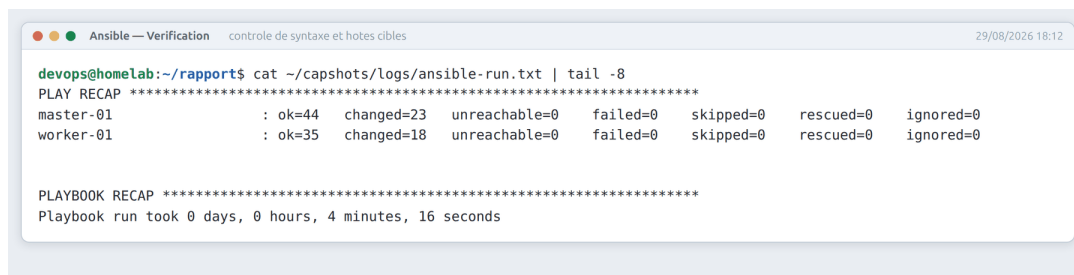
Un durcissement en deux temps

Cloud-init accorde `NOPASSWD:ALL` au compte `devops` le temps qu'Ansible installe des paquets. Le rôle `harden_sudo`, joué explicitement en fin de course (`-tags harden`), repose ensuite une politique `NOPASSWD` **restreinte** à une liste blanche (`kubeadm`, `kubelet`, quelques `systemctl restart`) — et non `PASSWD`, car aucun mot de passe n'existe pour ce compte : l'exiger verrouillerait définitivement le nœud hors de portée SSH. Il est gardé, comme `k8s_reset`, par un double verrou tag + opt-in : un `ansible-playbook playbook.yml` ordinaire doit pouvoir rejouer `docker/k8s_common` sur des nœuds déjà durcis (pour ajouter un worker plus tard) sans se verrouiller lui-même hors de portée.

Un dernier rôle, `k8s_reset`, permet une remise à zéro complète et destructive (`kubeadm reset -f`, purge CNI, flush iptables) : il est protégé par le même double opt-in (tag `never` et variable `reset_confirmed=true`), une destruction ne pouvant jamais survenir par erreur.

Justification du choix Ansible plutôt que ses alternatives

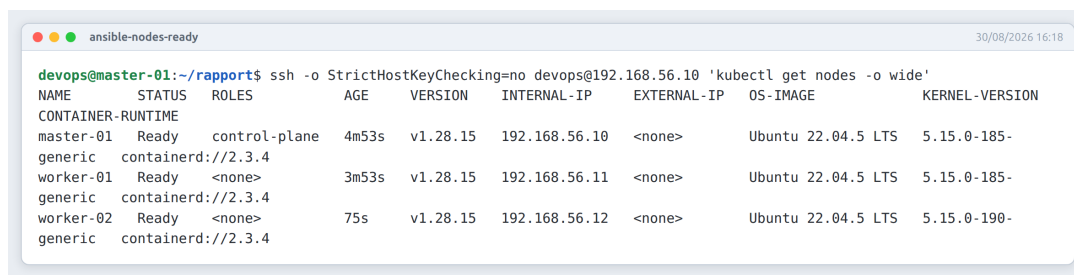
Ansible est agentless : rien à installer sur des VMs recréables à volonté par Terraform, seul SSH suffit. Kubespray aurait masqué la mécanique kubeadm — objectif pédagogique perdu ; k3s aurait caché les composants standard. Les choix fins (hold dpkg, no_log, addons différés, échec strict sur Calico) traduisent une pratique acquise par incidents réels, et non l'exécution d'un tutoriel.



```
devops@homelab:~/rapport$ cat ~/capshots/logs/ansible-run.txt | tail -8
PLAY RECAP *****
master-01      : ok=44  changed=23  unreachable=0  failed=0  skipped=0  rescued=0  ignored=0
worker-01     : ok=35  changed=18  unreachable=0  failed=0  skipped=0  rescued=0  ignored=0

PLAYBOOK RECAP *****
Playbook run took 0 days, 0 hours, 4 minutes, 16 seconds
```

Figure 2.4. Fin de run vert : PLAY RECAP rc=0, changed/unreachable, temps total (callback timer).



```
devops@master-01:~/rapport$ ssh -o StrictHostKeyChecking=no devops@192.168.56.10 'kubectl get nodes -o wide'
NAME          STATUS    ROLES    AGE   VERSION   INTERNAL-IP   EXTERNAL-IP   OS-IMAGE      KERNEL-VERSION
CONTAINER-RUNTIME
master-01     Ready    control-plane   4m53s   v1.28.15   192.168.56.10   <none>        Ubuntu 22.04.5 LTS   5.15.0-185-
generic      containerd://2.3.4
worker-01     Ready    <none>        3m53s   v1.28.15   192.168.56.11   <none>        Ubuntu 22.04.5 LTS   5.15.0-185-
generic      containerd://2.3.4
worker-02     Ready    <none>        75s    v1.28.15   192.168.56.12   <none>        Ubuntu 22.04.5 LTS   5.15.0-190-
generic      containerd://2.3.4
```

Figure 2.5. kubectl get nodes -o wide juste après le run : 3 nœuds Ready v1.28.x.

CHAPITRE 3

Orchestration : d'un commit à un pod en production

Le chapitre précédent a produit un cluster Kubernetes vide. Celui-ci suit le chemin inverse : comment le code applicatif devient-il une image, puis un pod en production, correctement dimensionné et isolé, sur le bon environnement ?

3.1 Construire : Docker et le modèle multi-stage

Docker sert exclusivement à **construire** les images dans ce projet ; l'exécution en cluster revient à `containerd`. Les trois services partagent rigoureusement le même modèle de Dockerfile :

```
FROM ${BASE_IMAGE} AS builder
WORKDIR /build
COPY shared/requirements.txt users-service/requirements.txt ./
RUN pip install -r shared-requirements.txt -r service-requirements.txt
COPY shared/ ./shared/
RUN pip install ./shared && rm -rf ../wheel* ../jaraco*

FROM ${BASE_IMAGE}
COPY --from=builder /usr/local/lib/python3.11/site-packages /usr/local/lib/python3
    .11/site-packages
RUN groupadd -r appuser && useradd -r -g appuser appuser --home-dir /app --no-
    create-home
COPY --chown=appuser:appuser users-service/main.py ./
USER appuser
HEALTHCHECK --interval=30s --timeout=3s CMD python -c "...
CMD ["python", "-m", "uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Listing 3.1. `app/users-service/Dockerfile` : structure multi-stage

Le stage *builder* contient `pip`, les caches et les outils de compilation ; l'image finale ne conserve que le strict runtime, exécuté par un utilisateur `appuser` non-root — aligné avec `runAsNonRoot` côté Kubernetes. Les trois images se construisent depuis un contexte commun (`app/`, jamais le dossier du service seul), pour que `shared/` soit importable comme un paquet racine.

Multi-stage : le standard des images de production

Séparer l'environnement de compilation de l'environnement d'exécution réduit à la fois la taille de l'image et sa surface d'attaque. Combiné à un utilisateur non-root, un HEALTHCHECK natif et des labels OCI de traçabilité, chaque image est directement scannable par Trivy (§6.1) sans bruit parasite.

3.2 Exécuter : containerd et la fin de Docker en production

Depuis la suppression de `dockershim` (Kubernetes 1.24), `containerd`, extrait de Docker et promu CNCF, est le chemin recommandé par `kubeadm` : plus léger, sans CLI ni API Docker exposées sur les nœuds. C'est lui qui télécharge les images depuis GHCR et crée les processus de conteneurs via `runc`. La bascule `SystemdCgroup = true` vue au chapitre précédent aligne `containerd` sur le comportement attendu par le kubelet 1.28 et évite une famille entière d'incidents : mémoire comptée deux fois, OOMKills injustifiés par un dual-cgrouping `cgroupfs/systemd`.

3.3 Orchestrer : Kubernetes 1.28 et Calico

Le control-plane, amorcé par `kubeadm` (chapitre 2), expose une API dont dépend tout le reste du cluster. Le réseau des pods repose sur **Calico v3.26.1** : au-delà du simple routage, Calico applique l'**intégralité** des NetworkPolicies — ingress *et* egress. C'est cette garantie, et non un détail technique, qui rend possible le modèle de sécurité réseau du chapitre 4 : sans elle, la stratégie « tout refuser par défaut puis lister les flux légitimes » serait inopérante. Un CNI dépourvu de politiques egress (Flannel, par exemple) aurait donc invalidé une section entière du modèle de sécurité de la plateforme.



```
Kubernetes — Connectivité réseau  résolution DNS et appel inter-pods 28/08/2026

devops@master-01:~/rapport$ kubectctl run netcheck --rm -i --restart=Never --image=busybox:1.36 -n devops-platform -- sh -c
'nslookup users-service.devops-platform.svc.cluster.local; wget -qO- --timeout=3 http://users-service/livez'
Error from server (Forbidden): pods "netcheck" is forbidden: violates PodSecurity "restricted:latest":
allowPrivilegeEscalation != false (container "netcheck" must set securityContext.allowPrivilegeEscalation=false),
unrestricted capabilities (container "netcheck" must set securityContext.capabilities.drop=["ALL"]), runAsNonRoot != true
(pod or container "netcheck" must set securityContext.runAsNonRoot=true), seccompProfile (pod or container "netcheck" must
set securityContext.seccompProfile.type to "RuntimeDefault" or "Localhost")
```

Figure 3.1. Test CNI : DNS + ping entre deux Pods de nœuds différents (`kubectctl exec + nslookup`).

3.4 Personnaliser par environnement : Kustomize

Kustomize personnalise des bundles YAML sans langage de template : un **base** décrit l'application générique, des **overlays** patchent uniquement les écarts par environnement.

```
k8s/apps/
  base/ # namespace, limites, HPA, PDB, RBAC, netpol
  users/ products/ orders/ # deployment + service + servicemonitor
  overlays/
    dev/ # replicas=1
    staging/ # namespace staging
    prod/ # namespace prod, replicas=3, PDB=2, digests
```

Listing 3.2. k8s/apps/ : organisation

Pour chacun des trois services, ce bundle produit environ dix objets Kubernetes (Deployment, Service, ServiceMonitor, HPA, PDB, ServiceAccount, NetworkPolicies partagées, Secret consommé), soit une trentaine d'objets pour la seule application — tous synchronisés par ArgoCD.

Le namespace **devops-platform** porte un profil **PSA restricted**, un **LimitRange** (défauts 250m CPU / 256 Mi, plafond 4 CPU / 2 Gi) et un **ResourceQuota** global. Chaque conteneur est construit avec le même socle de sécurité :

```
securityContext:
  runAsNonRoot: true
  seccompProfile: {type: RuntimeDefault}
containers:
  - securityContext:
      allowPrivilegeEscalation: false
      readOnlyRootFilesystem: true
      capabilities: {drop: ["ALL"]}
      startupProbe: {httpGet: {path: /livez, port: http}, failureThreshold: 30,
        periodSeconds: 5}
      readinessProbe: {httpGet: {path: /readyz, port: http}}
      livenessProbe: {httpGet: {path: /livez, port: http}}
```

Listing 3.3. Deployment users-service : extraits significatifs

La séparation **livenessProbe/readinessProbe** est un choix architectural délibéré, détaillé au chapitre 4 : la liveness ne vérifie que le processus, la readiness vérifie explicitement l'accessibilité de Vault — un pod vivant mais privé de secrets sort du Service plutôt que de servir dégradé silencieusement.

L'autoscaling suit un profil délibérément asymétrique : montée rapide, descente prudente.

Tableau 3.1. HPA — comportement anti-flapping

Direction	Réglage
Montée (scaleUp)	fenêtre de stabilisation 30 s, +100 % possibles toutes les 30 s
Descente (scaleDown)	fenêtre de stabilisation 300 s, −25 % par minute seulement

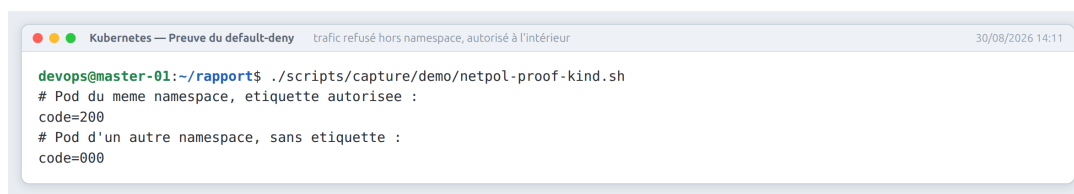
Les NetworkPolicies appliquent le même principe que le PSA : **default-deny** puis liste blanche explicite (sortie vers Vault sur 8200, DNS vers kube-system, entrée Prometheus sur 8000, trafic intra-namespace). Trois overlays adaptent ces bases à trois postures distinctes :

Aspect	dev	staging	prod
Réplicas	1	2 (base)	3
PDB	1	1	2
minAvailable			
Images	latest	latest	digest sha256 par CD
Secrets	replis dev actifs	Vault réel	Vault réel fail-closed

Tableau 3.2. Trois postures pour trois environnements

Pourquoi Kustomize plutôt que Helm ici

Kustomize patche sans langage de template : le **base** reste lisible et chaque environnement ne déclare que ses écarts ; il est natif dans **kubectl** et de première classe chez ArgoCD. Helm reste réservé aux stacks tierces (supervision) où le paramétrage justifie un vrai moteur de templates.



```

Kubernetes — Preuve du default-deny  trafic refusé hors namespace, autorisé à l'intérieur  30/08/2026 14:11
devops@master-01:~/rapport$ ./scripts/capture/demo/netpol-proof-kind.sh
# Pod du meme namespace, etiquette autorisee :
code=200
# Pod d'un autre namespace, sans etiquette :
code=000

```

Figure 3.2. Preuve default-deny : un Pod sans label ne peut joindre users-service, un Pod du namespace oui (**kubectl run test**).

CHAPITRE 4

Applications et gestion dynamique des secrets

4.1 Trois services, une seule plateforme

Les trois microservices FastAPI (*Users*, *Products*, *Orders*) partagent une structure rigoureusement identique — même `main.py`, mêmes dépendances, même Dockerfile — et ne diffèrent que par trois éléments : leur nom, leur endpoint métier et le secret spécifique qu'ils consomment.

Service	Endpoint métier	Secret propre (en plus de DATABASE_URL)
users-service	GET /users	JWT_SECRET_KEY
products-service	GET /products	API_KEY
orders-service	GET /orders	PAYMENT_GATEWAY_KEY

Ce n'est pas une simplification paresseuse : c'est le point du projet. La plateforme transverse — CI, secrets, monitoring — est ce qui compte, et elle est identique pour les trois services. Chacun expose quatre endpoints communs : / (métadonnées), /livez, /readyz, /metrics.

4.2 Le contrat fail-closed

Le cœur de la conception applicative tient en une règle : **un secret manquant en production doit faire échouer le démarrage, jamais servir une valeur de repli silencieuse.**

```
def _load_secrets() -> None:
    is_dev = os.environ.get("ENVIRONMENT", "production").lower() \
        in ("dev", "development", "local")

    try:
        DATABASE_URL = get_secret("DATABASE_URL")
    except SecretUnavailable:
```

```

DATABASE_URL = None
if not DATABASE_URL:
    if is_dev:
        DATABASE_URL = "sqlite:///file::memory:?cache=shared&uri=true"
        _LOG.warning("secret.default_used", extra={"secret_name": "DATABASE_URL"
            })
    else:
        raise SystemExit(
            "DATABASE_URL is missing in Vault and no env override is set; "
            "refusing to start in production without it.")

```

Listing 4.1. main.py — résolution des secrets au démarrage (fail-closed)

Les secrets sont résolus **à l'import du module**, pas au premier appel qui en aurait besoin : un problème Vault devient un crash immédiat et explicite au démarrage du pod, jamais une erreur 500 tardive côté client. En développement seulement, la plateforme accepte un repli assumé et journalisé (sqlite en mémoire, clé JWT éphémère générée par `secrets.token_hex(32)`) ; en production, l'absence du secret lève un `SystemExit` explicite.

Ce contrat se prolonge dans la readiness probe, qui dépend explicitement de la santé de Vault :

```

@app.get("/readyz")
def readyz():
    health = vault_health()
    return JSONResponse(
        status_code=200 if health.get("reachable") else 503,
        content={"service": "users", "vault": health},
    )

```

Listing 4.2. /readyz : sortir du Service plutôt que servir dégradé



Figure 4.1. Pod en crashloop après suppression du secret : `kubectl get pods + logs` montrant le `SystemExit` fail-closed.

4.3 La bibliothèque partagée : un seul comportement pour trois services

`app/shared/` est installé comme paquet Python dans chaque image (`pip install ./shared`) : cela garantit que les trois services partagent exactement la même politique de secrets, le même format de logs et la même configuration, avec un correctif unique pour les trois. Son module central, `vault_client.py` (239 lignes), implémente la résolution du token Vault selon un ordre de priorité explicite, conçu pour rester compatible avec la production :

```
def _vault_token() -> str:
    """1. /vault/secrets/token (Vault Agent Injector)
       2. VAULT_TOKEN env var (legacy/dev path)
       3. VAULT_DEV_ROOT_TOKEN_ID (dev mode only)"""
    injector_path = "/vault/secrets/token"
    if os.path.exists(injector_path):
        token = open(injector_path).read().strip()
        if token:
            return token
    token = os.environ.get("VAULT_TOKEN") or os.environ.get("
        VAULT_DEV_ROOT_TOKEN_ID")
    if not token:
        raise SecretUnavailable("No Vault token available.")
    return token
```

Listing 4.3. Résolution du token Vault (production-safe)

Chaque secret est lu au chemin KV-v2 `secret/data/devops-platform/<service>`, mis en cache pour la durée de vie du processus (`lru_cache`, invalidable par `reload_secrets()` pour la rotation), et **tout repli** — variable d’environnement ou valeur par défaut — est journalisé en **warning** : un fallback silencieux est structurellement impossible. Deux autres modules complètent le paquet : `log_config.py` (logs JSON structurés vers stdout, format consommé tel quel par ELK) et `config.py` (dataclass de configuration minimaliste).

Chaque service expose aussi ses métriques via `prometheus_fastapi_instrumentator`, en une seule ligne de configuration qui normalise l’histogramme de latence et regroupe les codes de statut :

```
Instrumentator(
    should_group_status_codes=True, should_ignore_untemplated=True,
    excluded_handlers=["/livez", "/readyz", "/metrics"],
).instrument(app).expose(app, endpoint="/metrics", include_in_schema=False)
```

Listing 4.4. Instrumentation Prometheus : identique sur les trois services

Ces métriques alimentent directement les règles SLO présentées au chapitre suivant.

4.4 Vault : distribuer les secrets sans les figer dans Git

HashiCorp Vault (**1.15.2**) centralise le stockage chiffré, la distribution contrôlée et la rotation des secrets — là où des Secrets Kubernetes seuls ne sont que du base64 non chiffré, sans rotation ni TTL ni audit. Son namespace porte un profil PSA **baseline** plutôt que *restricted*, pour une raison précise : Vault a besoin de la capacité noyau `IPC_LOCK` pour verrouiller sa mémoire.

Un mode dev assumé, un chemin de production tracé

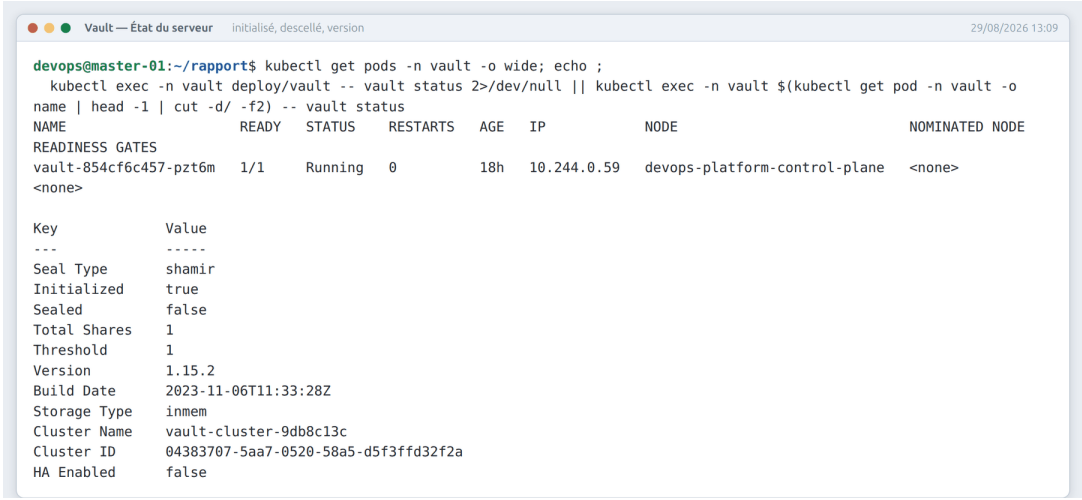
Le déploiement actuel est volontairement simple : un seul réplica en mode développeur (stockage en mémoire, auto-unseal). Ce choix de lab est assumé et documenté : il permet de démontrer dès aujourd'hui toute la mécanique applicative (KV-v2, politiques, rotation) sans porter encore la complexité opérationnelle de production. Cette limite est reprise et quantifiée au chapitre 8.

Un Job Kubernetes d'amorçage (`vault-setup`) configure Vault au premier lancement : moteur KV-v2, authentification Kubernetes, une politique par service appliquée depuis `vault-policy.hcl`. Cette politique applique le principe de moindre privilège de façon littérale :

```
path "secret/data/devops-platform/${svc}" { capabilities = ["read"] }
path "secret/metadata/devops-platform/${svc}" { capabilities = ["read", "list"] }
# KV v2 delete capability is intentionally NOT granted to the
# application ServiceAccount - applications must never delete their own secrets.
```

Listing 4.5. `k8s/vault/vault-policy.hcl` : intégralité utile

Deux capacités seulement, `read` sur la donnée et `read+list` sur les métadonnées : les applications consomment des secrets, elles ne les gèrent pas. Le token root lui-même n'est jamais écrit en clair sur disque ni versionné : il est généré ou roté *hors bande* par `scripts/bootstrap-vault-secret.sh`, passé **par `stdin`** — jamais en argument de ligne de commande, visible dans `ps` — et distribué via un Secret Kubernetes que les pods référencent avec `optional: false` : pas de Secret, pas de pod.



The terminal window shows the execution of `kubectl get pods -n vault -o wide; echo ;` followed by `kubectl exec -n vault deploy/vault -- vault status 2>/dev/null || kubectl exec -n vault $(kubectl get pod -n vault -o name | head -1 | cut -d/ -f2) -- vault status`. The output displays the pod status and the Vault configuration.

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED NODE
vault-854cf6c457-pzt6m	1/1	Running	0	18h	10.244.0.59	devops-platform-control-plane	<none>

Key	Value
Seal Type	shamir
Initialized	true
Sealed	false
Total Shares	1
Threshold	1
Version	1.15.2
Build Date	2023-11-06T11:33:28Z
Storage Type	inmem
Cluster Name	vault-cluster-9db8c13c
Cluster ID	04383707-5aa7-0520-58a5-d5f3ffd32f2a
HA Enabled	false

Figure 4.2. `kubectl get pods -n vault + vault status` exec dans le pod : initialized true, sealed false.

CHAPITRE 5

Observabilité et objectifs de niveau de service

Les métriques disent *qu'il y a* un problème ; les logs disent *pourquoi*. Ce chapitre traite les deux chaînes d'observabilité — métriques et logs — comme un seul système cohérent, alimentant à la fois des tableaux de bord et un contrat de service chiffré.

5.1 Prometheus : la collecte de métriques

Le Prometheus Operator (**v0.74.0**) transforme la configuration de Prometheus en ressources Kubernetes déclaratives. L'instance active (**v2.51.0**) scrape toutes les 15 secondes, retient 15 jours de données plafonnés à 8 GiB, et découvre ses cibles via des **ServiceMonitor** — chaque service publie le sien :

```
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
spec:
  selector: {matchLabels: {app.kubernetes.io/name: users-service}}
  endpoints:
    - {port: http, path: /metrics, interval: 15s, scrapeTimeout: 10s}
```

Listing 5.1. ServiceMonitor : intervalle 15 s aligné sur Prometheus

Ajouter un service supervisé se résume à poser un **ServiceMonitor** : exactement ce que GitOps sait synchroniser sans intervention. **kube-state-metrics** (v2.10.1) complète cette vue en exposant l'état des objets Kubernetes eux-mêmes — réplicas courants d'un HPA, raison de terminaison d'un pod — et constitue la source indispensable des alertes plateforme les plus critiques.

```
# Latence p95 par handler
histogram_quantile(0.95, sum by(le,handler)(rate(
  http_request_duration_seconds_bucket[5m])))
# OOMKills des dernieres 24h par pod
increase(kube_pod_container_status_last_terminated_reason{reason="OOMKilled"}[24h
])
# Disponibilite glissante 30 jours (recording rule)
```

```
service:availability_30d:ratio
```

Listing 5.2. Extrait du carnet de requêtes PromQL versionné

La contrainte réelle : tenir dans 12 Go de cluster

Sur les VMs à 4 Go, l'ensemble de la stack d'observabilité (Elasticsearch, Kibana, Logstash, Prometheus, ArgoCD, Grafana) occupe entre 4,3 et 5 Go de mémoire pour un cluster de 12 Go au total : une marge fine, qui explique les heaps JVM réduits, les queues bornées et le `allow_mmap: false` d'Elasticsearch documentés plus loin. C'est cette même contrainte qui a motivé, au chapitre 2, le passage de la RAM par nœud de 2 à 4 Go.

5.2 AlertManager : du seuil PromQL à l'alerte routée

AlertManager (**v0.27.0**) groupe et route les alertes définies dans un `PrometheusRule` unique (`rules.yaml`), qui contient à la fois des recording rules (pré-calcul des SLO) et des règles d'alerte :

```
- record: service:availability_30d:ratio
  expr: avg_over_time(up{job=~"users-service|products-service|orders-service"}[30d])
- alert: SLOAvailabilityBreach
  expr: (1 - service:availability_30d:ratio) * 100 > 0.1
  for: 10m
  labels: {severity: critical}
```

Listing 5.3. Recording rule + alerte disponibilité : extraits exacts

Six alertes composent le catalogue complet de la plateforme :

Tableau 5.1. Catalogue complet des alertes

Alerte	Condition	for	Gravité
SLOAvailabilityBreach	dispo. 30 j < 99,9 %	10 min	critical
SLOLatencyP95Breach	p95 > 200 ms	5 min	warning
SLO5xxErrorRateBreach	taux 5xx > 1 %	2 min	critical
ContainerMemoryRatioHigh	RAM > 80 % limite	2 min	warning
PodEvictionOngoing	OOMKill détecté	1 min	critical
HPAPinnedAtMaxReplicas	HPA au max 15 min	15 min	warning

Alerter sur ce qui est déjà arrivé

Chaque alerte de cette table correspond à un incident réellement rencontré pendant le développement (fuite mémoire, HPA bloqué au maximum) : la supervision n’anticipe pas des scénarios théoriques, elle formalise des échecs déjà vécus. Les preuves d’exécution de ces alertes font l’objet du chapitre 9.

5.3 Grafana : les tableaux de bord comme du code

Grafana 11.1.0 est provisionné entièrement en YAML — datasources (Prometheus et Elasticsearch) et quatre dashboards en ConfigMap, chargés par un sidecar (kiwigrid/k8s-sidecar) qui surveille en continu les ConfigMaps labellisés grafana_dashboard. Aucun export/import manuel de JSON : la vérité reste dans Git, comme tout autre manifest, et est synchronisée par ArgoCD.

Tableau 5.3. Les quatre dashboards provisionnés

Dashboard	Contenu
Infrastructure Overview	Nœuds Ready, total Pods, CPU/mémoire cluster %
Application Performance	RPS par handler, p95/p99, heatmap des durées
Error Rate SLO	% 5xx avec ligne de seuil 1 %, table des endpoints en erreur
Infrastructure Detail	Ratio mémoire (seuils 80/90), OOMKills, HPA au max

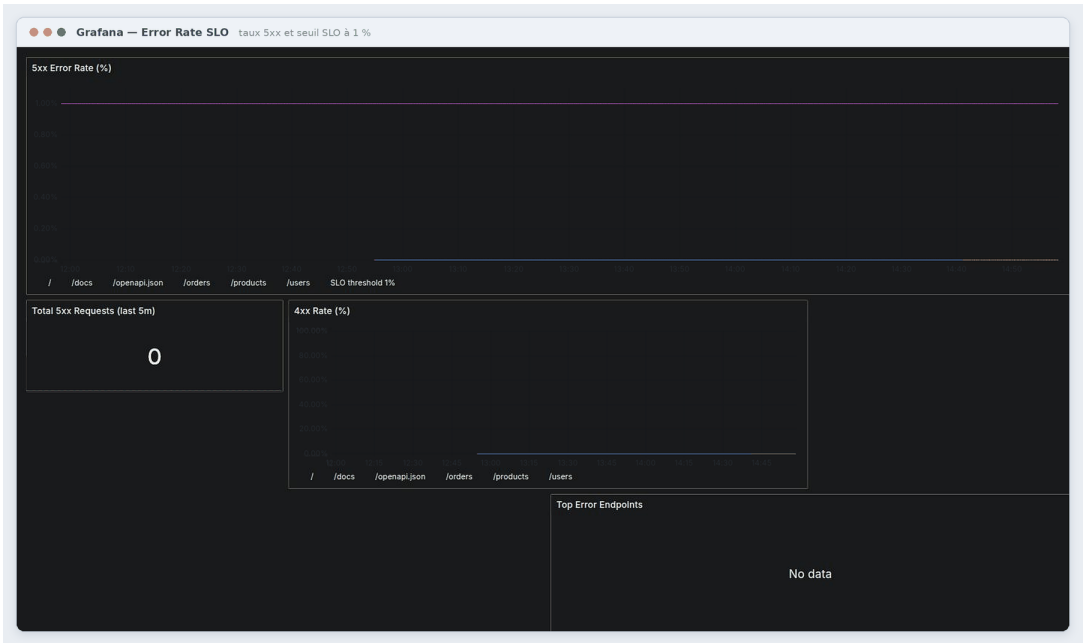


Figure 5.1. Dashboard Error Rate SLO avec ligne de seuil 1 % visible.

5.4 ELK : quand les métriques ne suffisent plus

Quatre composants Elasticsearch 8.14.0 forment la chaîne de logs centralisés : **Filebeat** en DaemonSet collecte sur chaque nœud, avec une autodiscovery conditionnée au label `part-of: devops-platform` — tout nouveau pod de la plateforme est loggé sans configuration supplémentaire ; **Logstash** enrichit le flux (ajout des champs `service/log_level`, et surtout un filtre qui *supprime* les requêtes de health-check pour ne pas noyer les logs utiles sous du bruit pur) ; **Elasticsearch** stocke et indexe ; **Kibana** explore.

```
filter {
  json { source => "message" target => "app" }
  if [url][path] in ["/livez", "/readyz"] { drop {} } # bruit pur
}
output { elasticsearch { index => "devops-platform-%{+YYYY.MM.dd}" } }
```

Listing 5.4. Pipeline Logstash : filtrage et enrichissement

Toute la pile est dimensionnée au plus juste pour tenir dans l'homelab : `ES_JAVA_OPTS -Xms512m/-Xmx512m`, `allow_mmap: false`, circuit breaker à 40 %, `NODE_OPTIONS -max-old-space-size=320` pour Kibana. Un script dédié, `bootstrap-elasticsearch-secret.sh`, crée les trois Secrets nécessaires (Elasticsearch, Kibana, Logstash) en refusant tout placeholder et en évitant `-from-literal` en ligne de commande.

5.5 Le modèle de menaces vu à travers l'observabilité

La supervision et la sécurité se recoupent : une alerte n'a de valeur que si elle détecte effectivement un vecteur d'attaque réel.

Tableau 5.5. Lecture croisée menace / contre-mesure observée

Menace	Vecteur	Contre-mesure en place
Secret exposé au cluster	Secrets K8s lisibles, tokens longs	Vault KV-v2 TTL 1h, politiques read-only, egress 8200 seule sortie
Conteneur privilégié	mauvais manifest	PSA restricted + conftest deny + Kyverno audit
Mouvement latéral	Pod compromis scanne le cluster	default-deny ingress/egress + whitelist explicite

Menace	Vecteur	Contre-mesure en place
Dérive manuelle du cluster	<code>kubectl edit</code> hors process	ArgoCD selfHeal + prune

5.6 SLO : transformer la supervision en contrat

Les seuils vus dans ce chapitre ne sont pas une slide isolée : ils existent à trois endroits cohérents — les règles Prometheus, la ligne de seuil du dashboard Error Rate, et ce rapport.

SLI	SLO
Disponibilité	$\geq 99,9\%$ sur 30 jours
Latence p95	< 200 ms
Taux d'erreurs 5xx	$< 1\%$
MTTD (délai de détection)	< 2 min
MTTR (délai de rétablissement)	< 30 min

Les valeurs de MTTD/MTTR découlent directement des intervalles techniques : scrape 15s + fenêtre `for` de 2 à 10 minutes impliquent une détection en moins de deux minutes ; l'auto-réparation Kubernetes combinée au selfHeal ArgoCD implique un rétablissement typique en quelques secondes à quelques minutes.

CHAPITRE 6

Intégration et déploiement continu

Ce chapitre suit le trajet complet d'un changement de code, du `git push` jusqu'au pod exécuté en production, en traitant l'intégration continue (push, GitHub Actions) et le déploiement continu (pull, ArgoCD) comme un seul flux cohérent plutôt que deux sujets séparés.

6.1 GitHub Actions : vérifier avant de publier

Un workflow unique (`.github/workflows/ci-cd.yml`, 447 lignes) gouverne toute la chaîne, avec des permissions minimales par défaut (`contents: read` au niveau racine, escalade par job au besoin) et une politique de concurrence qui annule les runs obsolètes sur une même référence.

```
lint ----> test --> build --> trivy-scan ----> deploy (manuel)
      | |
gitleaks----> terraform-validate -----+
load-test (schedule uniquement, independant)
```

Listing 6.1. Ordre et dépendances des jobs

Le job **lint** exécute Ruff (Python), `yamllint`, `terraform fmt`, et `kubeconform` en validation stricte hors ligne contre les schémas Kubernetes 1.28. Le job **gitleaks** scanne l'historique complet (`fetch-depth: 0`) et bloque le pipeline au moindre motif de secret détecté. Le job **test** exécute `pip-audit -strict` sur les quatre fichiers de dépendances — toute CVE connue casse le job — puis `pytest` et un build de fumée des trois images.

Le job **build** construit les trois images en matrice, avec cache GitHub Actions et attestations de provenance et de SBOM attachées à chaque image poussée. Le job **trivy-scan** (§6.1) exécute trois passes par service : rapport SARIF publié dans l'onglet Security de GitHub, gate bloquant sur les vulnérabilités CRITICAL/HIGH corrigibles, et export d'un SBOM SPDX archivé 30 jours. Enfin, le job **deploy**, seul point d'entrée du déploiement en production, n'est déclenché que manuellement (`workflow_dispatch`) : c'est la seule porte par laquelle un déploiement en production peut avoir lieu, l'humain restant délibérément dans la boucle.

Le pipeline se durcit lui-même

Actions toutes épinglées par version, permissions minimales par job, secrets scopés (GITHUB_TOKEN natif plutôt qu'un PAT), KUBECONFIG décodé en 0600 : le pipeline est traité comme une surface d'attaque à part entière, pas seulement comme un outil de productivité. Le chemin critique lint→test→build→trivy reste sous 15 minutes grâce au cache pip et buildx.

Un job nocturne (`schedule`, 3h du matin) rejoue `pip-audit` et Trivy sur le code déjà mergé : une CVE publiée *après* le merge fait échouer ce run sans qu'aucune ligne du dépôt n'ait changé — la détection de drift supply-chain présentée en scénario au chapitre 9.

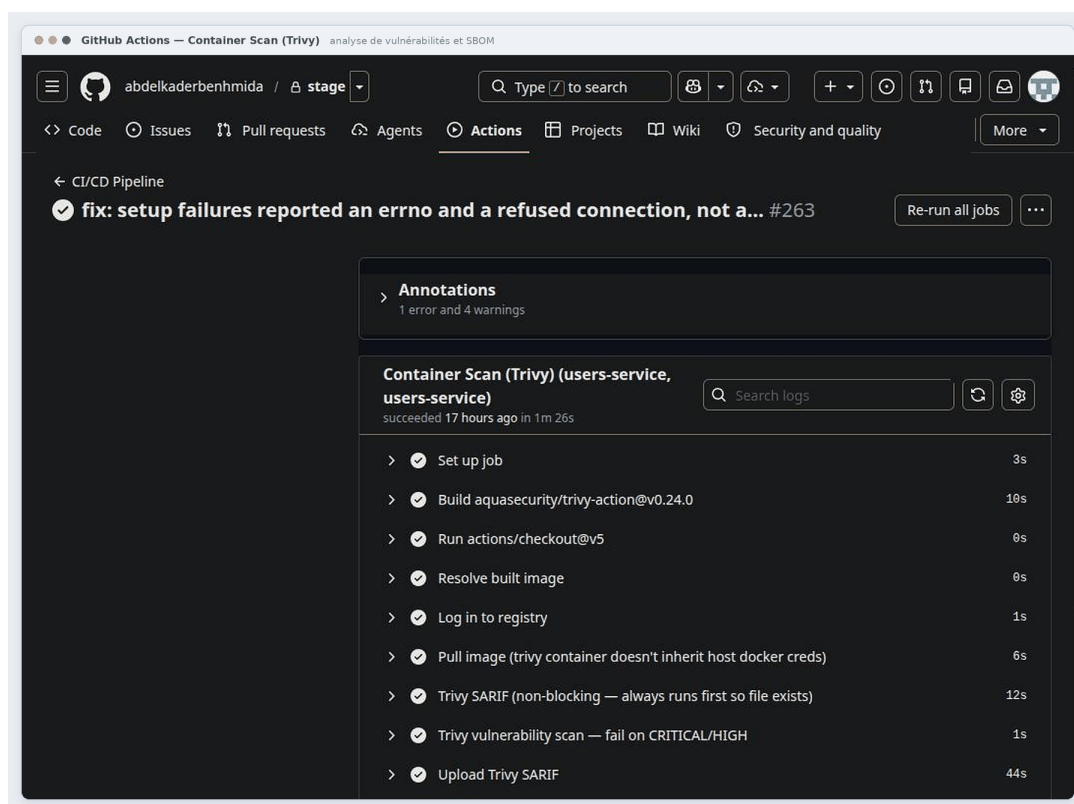


Figure 6.1. Job trivy-scan : table CRITICAL/HIGH vide + upload SARIF réussi.

6.2 ArgoCD : le cluster tire son état depuis Git

ArgoCD (**v3.5.1**) inverse le sens habituel du déploiement : le cluster *tire* son état désiré depuis Git au lieu de le recevoir d'une CI qui pousse. Douze **Application** couvrent l'ensemble de la plateforme, de son propre bootstrap (`argocd-install`, wave `-100`) jusqu'aux trois microservices, en passant par la chaîne d'observabilité complète.

La politique de synchronisation, commune aux douze **Applications**, tient en trois idées. D'abord, **automated** + **prune** + **selfHeal** : tout commit poussé s'applique

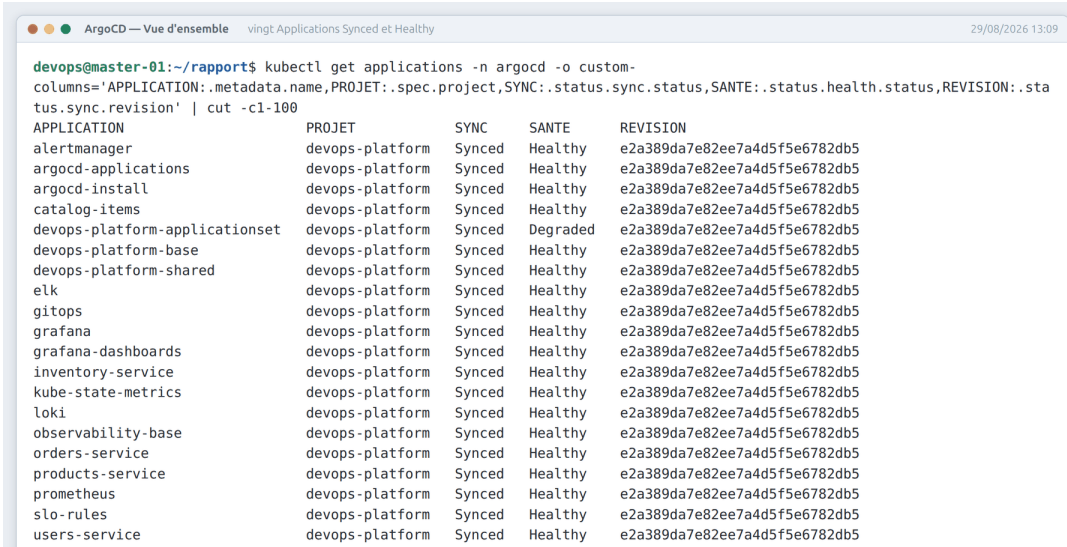
automatiquement, toute ressource disparue de Git est purgée du cluster, et toute modification manuelle hors Git (un `kubectl edit` improvisé) est annulée au cycle suivant — le cluster ne peut plus diverger silencieusement de Git. Ensuite, des options techniques soignées : `ServerSideApply` évite les collisions de champs, `PruneLast` garantit que les suppressions n'interviennent qu'une fois les créations saines. Enfin, l'ordre de démarrage est garanti par des `sync waves` : l'installation d'ArgoCD porte la vague -100, le socle monitoring la vague 5, Prometheus la 15, AlertManager/ELK la 20, Grafana la 25 — Grafana ne peut donc jamais démarrer avant que sa datasource Prometheus n'existe.

```
git push (nouveau tag image dans kustomization.yaml)
--> GitHub Actions: build + trivy OK --> GHCR (digest sha256)
--> ArgoCD detecte le nouveau commit
--> kustomize rend l'état desire, diff avec le cluster
--> Sync wave par sync wave: monitoring -> apps
--> Pods recreees sur les nouveaux digests, health check
```

Listing 6.2. Du commit au cluster : zéro `kubectl` humain

Ce que le GitOps change concrètement

Toute dérive est corrigée par `selfHeal` ; tout rollback est un `git revert` ; l'état complet du système est auditable dans l'historique Git. Les tags immuables (branche + sha court) et l'absence de `:latest` mutable hors branche par défaut rendent chaque déploiement retraçable jusqu'au commit exact — condition nécessaire à un rollback fiable, démontré en scénario au chapitre 9.



The screenshot shows the ArgoCD 'Overview' view with the title 'vingt Applications Synced et Healthy'. It displays a table of applications managed by ArgoCD. The table has five columns: APPLICATION, PROJET, SYNC, SANTE, and REVISION. All 12 applications listed are in a 'Synced' state and have a 'Healthy' status. The applications are: alertmanager, argocd-applications, argocd-install, catalog-items, devops-platform-applicationset, devops-platform-base, devops-platform-shared, elk, gitops, grafana, grafana-dashboards, inventory-service, kube-state-metrics, loki, observability-base, orders-service, products-service, prometheus, slo-rules, and users-service. All are associated with the 'devops-platform' project.

APPLICATION	PROJET	SYNC	SANTE	REVISION
alertmanager	devops-platform	Synced	Healthy	e2a389da7e82ee7a4d5f5e6782db5
argocd-applications	devops-platform	Synced	Healthy	e2a389da7e82ee7a4d5f5e6782db5
argocd-install	devops-platform	Synced	Healthy	e2a389da7e82ee7a4d5f5e6782db5
catalog-items	devops-platform	Synced	Healthy	e2a389da7e82ee7a4d5f5e6782db5
devops-platform-applicationset	devops-platform	Synced	Degraded	e2a389da7e82ee7a4d5f5e6782db5
devops-platform-base	devops-platform	Synced	Healthy	e2a389da7e82ee7a4d5f5e6782db5
devops-platform-shared	devops-platform	Synced	Healthy	e2a389da7e82ee7a4d5f5e6782db5
elk	devops-platform	Synced	Healthy	e2a389da7e82ee7a4d5f5e6782db5
gitops	devops-platform	Synced	Healthy	e2a389da7e82ee7a4d5f5e6782db5
grafana	devops-platform	Synced	Healthy	e2a389da7e82ee7a4d5f5e6782db5
grafana-dashboards	devops-platform	Synced	Healthy	e2a389da7e82ee7a4d5f5e6782db5
inventory-service	devops-platform	Synced	Healthy	e2a389da7e82ee7a4d5f5e6782db5
kube-state-metrics	devops-platform	Synced	Healthy	e2a389da7e82ee7a4d5f5e6782db5
loki	devops-platform	Synced	Healthy	e2a389da7e82ee7a4d5f5e6782db5
observability-base	devops-platform	Synced	Healthy	e2a389da7e82ee7a4d5f5e6782db5
orders-service	devops-platform	Synced	Healthy	e2a389da7e82ee7a4d5f5e6782db5
products-service	devops-platform	Synced	Healthy	e2a389da7e82ee7a4d5f5e6782db5
prometheus	devops-platform	Synced	Healthy	e2a389da7e82ee7a4d5f5e6782db5
slo-rules	devops-platform	Synced	Healthy	e2a389da7e82ee7a4d5f5e6782db5
users-service	devops-platform	Synced	Healthy	e2a389da7e82ee7a4d5f5e6782db5

Figure 6.2. UI ArgoCD : tuiles des 12 Applications toutes Synced/Healthy.

```

devops@master-01:~/rapport$ echo '# derive manuelle : passage a 5 replicas hors Git'; kubectl scale deploy/users-service -
n devops-platform --replicas=5; for i in 1 2 3 4 5 6 7 8; do printf '%s spec.replicas=%s sync=%s\n' "$(date +%H:%M:%S)"
"$(kubectl get deploy users-service -n devops-platform -o jsonpath='{.spec.replicas}')" "$(kubectl get app users-
service -n argocd -o jsonpath='{.status.sync.status}');" sleep 8; done; echo; echo '# ArgoCD a ramene l etat declare dans
Git'
# derive manuelle : passage a 5 replicas hors Git
deployment.apps/users-service scaled
15:22:43 spec.replicas=5 sync=0out0fSync
15:22:51 spec.replicas=5 sync=0out0fSync
15:22:59 spec.replicas=5 sync=0out0fSync
15:23:07 spec.replicas=5 sync=0out0fSync
15:23:15 spec.replicas=5 sync=0out0fSync
15:23:23 spec.replicas=5 sync=0out0fSync
15:23:31 spec.replicas=5 sync=0out0fSync
15:23:39 spec.replicas=5 sync=0out0fSync

# ArgoCD a ramene l etat declare dans Git

```

Figure 6.3. Démonstration selfHeal : `kubectl scale deploy users-service --replicas=5` manuel puis retour automatique à 2 par ArgoCD (avant/après).

6.3 Les contrôles DevSecOps intégrés au flux

Au-delà de Gitleaks et Trivy déjà vus, trois autres familles de contrôles ferment la boucle de sécurité du pipeline. **pre-commit** exécute localement les mêmes onze hooks que la CI (ruff, yamllint, gitleaks, terraform fmt/validate) à chaque commit, éliminant l’aller-retour « push → échec CI → fix ». **conftest** (politiques Rego) et **Kyverno** (politiques d’admission cluster) appliquent la même triple règle — `readOnlyRootFilesystem`, capacités `drop: ALL, runAsNonRoot` — à deux instants différents : à la validation CI pour conftest, à l’admission du cluster pour Kyverno. Les deux tournent aujourd’hui en mode **audit**, non bloquant : la police observe avant de sanctionner, trajectoire industrielle classique pour mesurer le bruit avant de bloquer (reprise comme limite assumée au chapitre 8).

```

deny[msg] {
  input.kind == "Deployment"
  container := input.spec.template.spec.containers[_]
  not container.securityContext.readOnlyRootFilesystem
  msg := sprintf("Container %s in %s must set readOnlyRootFilesystem: true",
    [container.name, input.metadata.name])
}

```

Listing 6.3. `security.rego` : une des trois règles conftest

```

devops@homelab:~/rapport$ kubectl kustomize k8s/apps/base | conftest test --policy k8s/policies/conftest -
60 tests, 60 passed, 0 warnings, 0 failures, 0 exceptions

```

Figure 6.4. `kustomize build k8s/apps/base | conftest test --policy k8s/policies/conftest -` : aucun refus sur les manifests conformes.

Partie II

Analyse critique

CHAPITRE 7

Choix techniques : synthèse comparative

Les chapitres précédents ont justifié chaque choix technique dans son propre contexte. Ce chapitre prend du recul et confronte ces choix de façon transversale : quelles familles d'alternatives ont été systématiquement écartées, et pourquoi.

7.1 Trois familles d'alternatives récurrentes

Tableau 7.1. Alternatives structurantes écartées et raison commune

Décision retenue		Alternatives écartées	Raison transversale
Vraies (libvirt/KVM)	VM	kind, minikube, Vagrant	Apprentissage réseau/systemd réel ; Vagrant redondant une fois Terraform en place
Cloud public partout	écarté	AWS/GCP, Secrets Manager, Elastic Cloud	Coût et facturation permanents contraires au principe homelab ; couplage propriétaire
Agentless qu'agents	plutôt	SaltStack, Chef, Puppet, kubespray	Cohérent avec des VM recréables à volonté ; kubespray aurait masqué la mécanique kubeadm
Format déclaratif sans templating		Pulumi (langage général), Helm pour l'app	Lisibilité et review directe du diff Git ; Helm réservé aux stacks tierces paramétrées
Scanners unifiés	CNCF	Snyk, scanners propriétaires multiples	Un seul outil (Trivy) couvre CVE + SBOM + SARIF nativement intégré GitHub

Décision retenue	Alternatives écartées	Raison transversale
Secrets dynamiques	Secrets K8s seuls, Sealed Secrets/SOPS	Rotation, TTL et audit imposent un vrai moteur de secrets, pas un simple chiffrement statique

7.2 Le fil directeur de ces arbitrages

Trois critères reviennent systématiquement dans les choix retenus, indépendamment de la couche concernée. Le premier est la **reproductibilité pédagogique** : un outil qui masque la mécanique sous-jacente (kind, kubespray, k3s) est systématiquement écarté au profit d'un outil qui l'expose, même au prix d'une configuration plus verbeuse — c'est un projet de fin d'études, pas un déploiement de production où la vitesse d'installation prime. Le second est la **contrainte de coût et d'autonomie** du homelab : tout service cloud facturé est remplacé par son équivalent auto-hébergé, y compris quand l'équivalent managé serait objectivement plus simple à opérer. Le troisième est la **traçabilité Git** : à choix technique équivalent, celui qui se relit et se revoit comme du texte (HCL, YAML patché par Kustomize, politiques Rego) est préféré à celui qui nécessite un outillage spécifique pour être audité.

Ces trois critères entrent parfois en tension. Le choix d'ELK plutôt que Loki en est l'exemple le plus net : Loki serait plus léger et plus proche de la contrainte mémoire réelle du homelab (chapitre 5), mais il aurait sacrifié la recherche plein-texte et l'enrichissement de pipeline que le critère pédagogique exigeait de pouvoir démontrer. La stack ELK a donc été retenue *malgré* la pression mémoire qu'elle impose, au prix des réglages JVM serrés documentés au chapitre 5 — un arbitrage assumé plutôt qu'un compromis subi sans le savoir.

CHAPITRE 8

Limites assumées et dette technique

Une lecture honnête du code actuel fait apparaître des limites réelles, chacune documentée dans le code lui-même plutôt que découverte a posteriori. Aucune ne remet en cause l'architecture d'ensemble, mais elles doivent être énoncées sans détour.

8.1 Dette de sécurité assumée

Vault en mode développeur. Le stockage est en mémoire, le descellement automatique, et le token racine partagé — un choix de lab délibéré (chapitre 4) mais qui ne saurait constituer une configuration de production en l'état. Le chemin de montée en gamme (stockage raft répliqué, TLS interne, unseal Shamir, injection par agent) est écrit mais non câblé.

Politiques d'admission en mode audit. `conftest` et `Kyverno` (§6.1) observent sans bloquer : un manifest non conforme au socle de sécurité peut aujourd'hui encore être admis dans le cluster. Le passage en mode **Enforce** nécessite d'abord une période d'observation pour écarter les faux positifs.

8.2 Dette d'exécution

- `tests/k6/load-test.js` n'existe pas : le job de test de charge nocturne échoue systématiquement à chaque exécution planifiée — c'est la priorité corrective la plus immédiate du projet ;
- le job `deploy` lit une seule valeur de sortie (`needs.build.outputs.image`) pour les trois lignes de la matrice de build : la dernière image construite écrase les précédentes dans cette variable, limitation documentée inline dans le workflow ;
- le job `trivy-scan` résout les images par tag `github.ref_name` et non par digest, malgré l'intention affichée d'un pinning strict ;
- l'overlay `dev` ramène les réplicas à 1 sans retirer le HPA associé (min 2) : un conflit latent entre l'intention de l'overlay et la configuration d'autoscaling.

8.3 Dette cosmétique et résiduelle

- l'AppProject ArgoCD conserve des destinations `istio-system/flagger-system` héritées d'une stack canary depuis retirée du projet ;
- le commentaire d'en-tête de Filebeat mentionne un envoi vers Logstash:5044, alors que la configuration active écrit directement vers Elasticsearch ;
- le checksum du manifest Tigera n'est pas vérifié (TODO inline) ; le `host_key_checking=False` d'Ansible est un choix assumé de lab, à ne jamais reconduire tel quel hors de ce contexte.

Pourquoi lister ces limites plutôt que les corriger silencieusement

Chaque point ci-dessus est borné, actionnable, et déjà signalé dans le code concerné par un commentaire ou un TODO. Cette double visibilité — ce qui marche, ce qui manque, pourquoi — est un choix méthodologique délibéré du projet, repris dans la conclusion générale : elle distingue un exercice d'ingénierie honnête d'un assemblage de tutoriels qui ne s'avoue jamais incomplet.

CHAPITRE 9

Validation : scénarios de bout en bout

Les chapitres précédents affirment que la plateforme se comporte d'une certaine façon face à un déploiement, une panne Vault, une fuite mémoire ou une régression. Ce chapitre transforme chacune de ces affirmations en **scénario rejouable** : objectif, étapes exactes, commandes, résultat observable. Ce sont ces démonstrations qui donnent sa valeur de preuve au reste du rapport.

9.1 Scénario 1 — Du commit au déploiement

Objectif. Prouver la chaîne complète décrite au chapitre 6 : un simple `git push` traverse lint, scans, build, registre, ArgoCD, jusqu'aux pods mis à jour, sans aucun `kubectl` humain.

```
git commit -m "chore: bump users image" && git push
gh run watch
kubectl get deploy users-service -o jsonpath='{.spec.template.spec.containers[0].
  image}'
curl -s localhost:18080/metrics | grep http_requests_total
```

Listing 9.1. Commandes d'observation du scénario

Étapes observées : modification bénigne dans le kustomization d'un service, run CI déclenché par le filtre de chemins, jobs successifs jusqu'à trivy-scan, nouvelle image visible dans GHCR, Application ArgoCD passant OutOfSync puis synchronisée vague par vague, nouveau digest confirmé côté cluster, métriques répondant sur le nouveau pod.

9.2 Scénario 2 — Incident Vault indisponible

Objectif. Démontrer le fail-closed de bout en bout (chapitre 4) : perte de Vault ⇒ readiness 503 ⇒ sortie du Service ⇒ zéro trafic dégradé ⇒ rétablissement propre.

```
kubectl scale deploy vault -n vault --replicas=0
kubectl get endpoints users-service -n devops-platform # vide !
kubectl logs deploy/users-service | grep vault.fetch_failed
```

```
kubectl scale deploy vault -n vault --replicas=1 # remediation
```

Listing 9.2. Boîte à outils de l'incident

Résultat observé : la readiness probe échoue après quelques cycles, le pod sort du Service (endpoints vides), mais la liveness probe reste **alive** — le processus ne redémarre pas inutilement, exactement le comportement voulu par la séparation liveness/readiness du chapitre 3. Une alerte **SLOAvailabilityBreach** se déclenche si la panne se prolonge.

9.3 Scénario 3 — Fuite mémoire et OOMKill

Objectif. Suivre la chaîne de protection mémoire du chapitre 5 : **LimitRange** → alerte à 80 % → **OOMKill** → auto-réparation.

En réduisant temporairement la limite mémoire d'un déploiement puis en générant une pression mémoire artificielle dans le pod, on observe successivement : le ratio mémoire franchir 80 % sur le dashboard Infrastructure Detail, l'alerte **ContainerMemoryRatioHigh** se déclencher dans **AlertManager**, puis le kubelet provoquer un **OOMKill** (**Last State: OOMKilled, exit code 137**) et l'alerte **PodEvictionOngoing**. C'est cet incident, réellement rencontré pendant le développement, qui a directement motivé le dimensionnement mémoire documenté au chapitre 1.

9.4 Scénario 4 — Stress HPA et anti-flapping

Objectif. Valider le comportement asymétrique de l'autoscaling (chapitre 3) : montée rapide sous charge, descente lente et prudente.

`scripts/stress-hpa.sh` génère une charge contrôlée (`ab -c 80 -n 4000`). Les répliques montent de 2 à 5 dès que le CPU dépasse 70 %, avec une fenêtre de stabilisation de 30 secondes ; à l'arrêt de la charge, la descente reste bloquée pendant 300 secondes avant de redescendre de 25 % par minute. Une charge maintenue plus de 15 minutes déclenche **HPAPinnedAtMaxReplicas**.

9.5 Scénario 5 — Rollback GitOps après régression

Objectif. Montrer qu'un rollback en production est un simple `git revert`, sans aucune manipulation directe du cluster.

```
git revert --no-edit <sha-regression>
git push
```

```
# ArgoCD fait le reste; historique complet dans l'UI
```

Listing 9.3. Rollback = git revert

Une régression volontaire (endpoint renvoyant systématiquement 500) fait franchir le seuil de 1 % du dashboard Error Rate SLO et déclenche `SL05xxErrorRateBreach`. Le `git revert` suffit : ArgoCD resynchronise vers le digest précédent et l'erreur disparaît, MTTR mesuré en minutes et entièrement traçable dans l'historique Git.

9.6 Scénario 6 — Rotation complète des secrets

Objectif. Prouver que la rotation d'un secret critique (chapitre 4) est une opération de routine : zéro downtime visible, zéro fuite du nouveau token.

```
scripts/bootstrap-vault-secret.sh
vault kv put secret/devops-platform/users-service JWT_SECRET_KEY="$(openssl rand -
  hex 32)"
kubectrl rollout restart deploy/users-service -n devops-platform
gitleaks detect --source . --no-banner # rien dans l'historique
```

Listing 9.4. Séquence de rotation propre

Le token n'est affiché qu'une seule fois par le script de bootstrap ; le rollout restart recharge les secrets au boot (modèle startup-load assumé) ; `readyz` repasse à 200 partout et aucune trace du nouveau secret n'apparaît dans l'historique Git ou shell.

9.7 Scénario 7 — Détection de drift supply-chain

Objectif. Montrer pourquoi le cron nocturne du chapitre 6 existe : une CVE publiée après le merge doit être détectée sans nouvelle livraison de code.

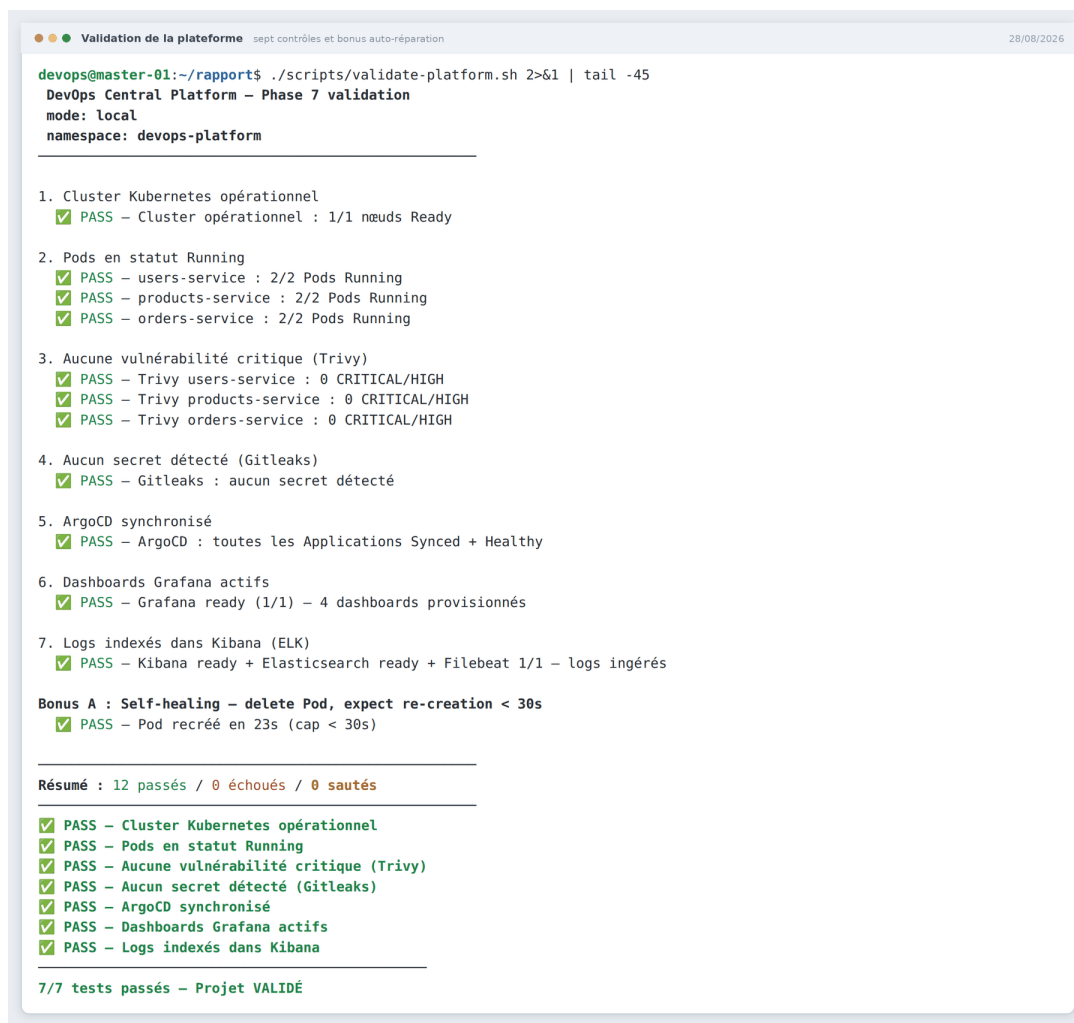
La sécurité comme processus, pas comme état

Ce qui était vert hier peut être rouge aujourd'hui sans qu'aucune ligne du projet n'ait changé. Le scan nocturne (`pip-audit`, Trivy) transforme cette réalité en signal automatique : un run rouge à 3h du matin est le MTDD de la supply chain, suivi d'une PR de bump de version classique qui repasse toute la chaîne de validation.

9.8 Outils de vérification continue

Au-delà des sept scénarios, deux scripts encodent en permanence la définition de « ça marche » et servent à la fois de recette locale, de garde CI et de preuve de démonstration.

`validate-platform.sh` exécute sept contrôles (cluster opérationnel, pods en Running, absence de CVE critique, absence de secret détecté, ArgoCD synchronisé, dashboards Grafana actifs, logs indexés dans ELK) plus un contrôle bonus de self-healing qui supprime volontairement un pod et vérifie son retour en moins de 30 secondes — seule façon de prouver que l’auto-réparation fonctionne réellement, et non seulement que sa configuration est présente. `validate-security.sh` vérifie en complément l’absence de secrets, l’absence de CVE, l’état descellé de Vault et la validité du token racine.



```

Validation de la plateforme sept contrôles et bonus auto-réparation 28/08/2026

devops@master-01:~/rapport$ ./scripts/validate-platform.sh 2>&1 | tail -45
DevOps Central Platform – Phase 7 validation
mode: local
namespace: devops-platform

1. Cluster Kubernetes opérationnel
  ✓ PASS – Cluster opérationnel : 1/1 nœuds Ready

2. Pods en statut Running
  ✓ PASS – users-service : 2/2 Pods Running
  ✓ PASS – products-service : 2/2 Pods Running
  ✓ PASS – orders-service : 2/2 Pods Running

3. Aucune vulnérabilité critique (Trivy)
  ✓ PASS – Trivy users-service : 0 CRITICAL/HIGH
  ✓ PASS – Trivy products-service : 0 CRITICAL/HIGH
  ✓ PASS – Trivy orders-service : 0 CRITICAL/HIGH

4. Aucun secret détecté (GitLeaks)
  ✓ PASS – GitLeaks : aucun secret détecté

5. ArgoCD synchronisé
  ✓ PASS – ArgoCD : toutes les Applications Synced + Healthy

6. Dashboards Grafana actifs
  ✓ PASS – Grafana ready (1/1) – 4 dashboards provisionnés

7. Logs indexés dans Kibana (ELK)
  ✓ PASS – Kibana ready + Elasticsearch ready + Filebeat 1/1 – logs ingérés

Bonus A : Self-healing – delete Pod, expect re-creation < 30s
  ✓ PASS – Pod recréé en 23s (cap < 30s)

Résumé : 12 passés / 0 échoués / 0 sautés

  ✓ PASS – Cluster Kubernetes opérationnel
  ✓ PASS – Pods en statut Running
  ✓ PASS – Aucune vulnérabilité critique (Trivy)
  ✓ PASS – Aucun secret détecté (GitLeaks)
  ✓ PASS – ArgoCD synchronisé
  ✓ PASS – Dashboards Grafana actifs
  ✓ PASS – Logs indexés dans Kibana

7/7 tests passés – Projet VALIDÉ

```

Figure 9.1. `scripts/validate-platform.sh` complet : les 7 contrôles + bonus en vert avec durées.

Conclusion générale et perspectives

Bilan du travail réalisé

DevOps Central Platform démontre qu'une chaîne DevOps professionnelle complète peut tenir dans un homelab : Terraform provisionne, cloud-init durcit, Ansible installe, Docker empaquette, Kubernetes orchestre, GitHub Actions vérifie et pousse, Trivy/Gitleaks/pip-audit filtrent la supply chain, Vault distribue les secrets en fail-closed, ArgoCD maintient Git comme unique source de vérité, Prometheus/Grafana/ELK rendent le tout observable et les SLO le rendent contractuel.

Trois enseignements traversent ce rapport :

1. **Chaque outil a été choisi pour une raison précise**, opposable aux alternatives : et cette justification est documentée outil par outil dans les encadrés dédiés ;
2. **La sécurité est transversale, pas additive** : elle apparaît au boot (cloud-init), dans les manifests (PSA, netpol, securityContext), dans la CI (scans bloquants), dans l'applicatif (fail-closed) et à l'admission (Kyverno) ;
3. **L'honnêteté technique fait partie du design** : les limites sont listées, les incidents ont leurs alertes, les TODO sont inline. C'est cette double visibilité (ce qui marche, ce qui manque, pourquoi) qui distingue un exercice d'ingénierie d'un assemblage de tutoriels.

Apport personnel

Au-delà des livrables techniques, ce projet a constitué une mise en situation d'ingénierie complète. Il a fallu arbitrer entre des solutions concurrentes à chaque étape, assumer ces arbitrages par écrit, puis les confronter à l'exécution réelle, qui a régulièrement invalidé les hypothèses initiales. Le dimensionnement mémoire des nœuds, la temporisation des add-ons Kubernetes ou la stratégie de gestion des secrets sont autant de points où la solution finale diffère de la solution envisagée au départ, précisément parce qu'elle a été éprouvée.

Ce travail m'a également permis de consolider des compétences transversales : la lecture de documentation technique de référence, la conduite d'un diagnostic méthodique en situation d'incident, et la rédaction d'une documentation d'exploitation destinée à une personne autre que son auteur.

Limites du travail

Plusieurs limites doivent être énoncées honnêtement. HashiCorp Vault fonctionne en mode *dev*, non persistant et déscellé automatiquement : ce mode convient à une démonstration mais ne saurait constituer une configuration de production. Les politiques d'admission Kyverno restent en mode *Audit* et ne bloquent donc pas encore les manifests non conformes. Le test de charge k6 est prévu par le pipeline mais son script n'est pas encore écrit. Enfin, la plateforme s'exécute sur un homelab à ressources contraintes : les chiffres de performance mesurés ne sont pas transposables tels quels à une infrastructure de production.

Perspectives d'évolution

Quatre axes de durcissement prolongent naturellement ce travail.

Sécurisation de la gestion des secrets. Passer Vault en mode production avec stockage persistant, descelllement à plusieurs parts, authentification Kubernetes par ServiceAccount plutôt que par token racine, et rotation automatisée des secrets.

Passage des politiques en mode bloquant. Après une période d'observation permettant d'identifier les faux positifs, basculer Kyverno en `validationFailureAction: Enforce` et étendre le jeu de règles aux labels obligatoires et aux *requests/limits* de ressources.

Consolidation de la chaîne d'intégration continue. Rédiger le scénario de test de charge k6, épingler l'analyse Trivy sur le digest issu de la construction plutôt que sur le tag, et adopter l'authentification OIDC pour le backend Terraform distant, dont le contrat est déjà écrit.

Enrichissement de la charge applicative. Substituer une base PostgreSQL réelle au stockage en mémoire (la NetworkPolicy correspondante est déjà réservée) et activer effectivement la vérification des jetons JWT dans les services.

À plus long terme, l'architecture se prête à une extension multi-cluster avec déploiement progressif (*canary* ou *blue-green*) piloté par ArgoCD Rollouts, ainsi qu'à l'ajout d'un troisième pilier d'observabilité, le traçage distribué, qui compléterait les métriques et les logs déjà en place.

Bibliographie

- [1] G. Kim, J. Humble, P. Debois, J. Willis, *The DevOps Handbook: How to Create World-Class Agility, Reliability, and Security in Technology Organizations*, IT Revolution Press, 2016.
- [2] J. Humble, D. Farley, *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*, Addison-Wesley, 2010.
- [3] B. Beyer, C. Jones, J. Petoff, N. R. Murphy, *Site Reliability Engineering: How Google Runs Production Systems*, O'Reilly Media, 2016.
- [4] B. Burns, J. Beda, K. Hightower, *Kubernetes: Up and Running*, 2^e édition, O'Reilly Media, 2019.
- [5] Y. Brikman, *Terraform: Up and Running: Writing Infrastructure as Code*, 3^e édition, O'Reilly Media, 2022.
- [6] K. Morris, *Infrastructure as Code: Dynamic Systems for the Cloud Age*, 2^e édition, O'Reilly Media, 2020.
- [7] J. Turnbull, *Monitoring with Prometheus*, Turnbull Press, 2018.
- [8] The Kubernetes Authors, *Kubernetes Documentation*, <https://kubernetes.io/docs/>, consulté en 2026.
- [9] The Kubernetes Authors, *Creating a cluster with kubeadm*, <https://kubernetes.io/docs/setup/production-environment/tools/kubeadm/>, consulté en 2026.
- [10] The Kubernetes Authors, *Pod Security Admission*, <https://kubernetes.io/docs/concepts/security/pod-security-admission/>, consulté en 2026.
- [11] HashiCorp, *Terraform Documentation*, <https://developer.hashicorp.com/terraform/docs>, consulté en 2026.
- [12] HashiCorp, *Vault Documentation*, <https://developer.hashicorp.com/vault/docs>, consulté en 2026.
- [13] Red Hat, *Ansible Documentation*, <https://docs.ansible.com/>, consulté en 2026.
- [14] Docker Inc., *Dockerfile best practices*, <https://docs.docker.com/build/building/best-practices/>, consulté en 2026.

- [15] Tigera, *Calico Documentation*, <https://docs.tigera.io/calico/latest/about/>, consulté en 2026.
- [16] Argo Project, *Argo CD: Declarative GitOps CD for Kubernetes*, <https://argo-cd.readthedocs.io/>, consulté en 2026.
- [17] OpenGitOps Working Group (CNCF), *GitOps Principles v1.0*, <https://opengitops.dev/>, consulté en 2026.
- [18] Prometheus Authors, *Prometheus Documentation*, <https://prometheus.io/docs/>, consulté en 2026.
- [19] Elastic, *Elastic Stack Documentation*, <https://www.elastic.co/guide/>, consulté en 2026.
- [20] S. Ramírez, *FastAPI Documentation*, <https://fastapi.tiangolo.com/>, consulté en 2026.
- [21] Aqua Security, *Trivy Documentation*, <https://aquasecurity.github.io/trivy/>, consulté en 2026.
- [22] OWASP Foundation, *OWASP Top 10 (2021)*, <https://owasp.org/Top10/>, consulté en 2026.
- [23] NIST, *Application Container Security Guide*, Special Publication 800-190, 2017.
- [24] Center for Internet Security, *CIS Kubernetes Benchmark*, <https://www.cisecurity.org/benchmark/kubernetes>, consulté en 2026.

ANNEXE A

Commandes utiles

```
# Cycle infra
cd terraform && terraform init && terraform apply
scripts/generate-inventory.sh # ou --no-refresh
cd ../ansible && ansible-playbook playbook.yml

# Deploiement cluster
kubectl apply -f k8s/apps/base/namespace.yaml
kubectl apply -f k8s/vault/manifests.yaml
scripts/bootstrap-vault-secret.sh
kubectl apply -f k8s/apps/
kubectl apply -k k8s/argocd/install/

# Validation
scripts/validate-platform.sh # resume
scripts/validate-platform.sh --ci # bloquant
scripts/validate-platform.sh --skip-incident
scripts/validate-security.sh

# Tests locaux
ENVIRONMENT=dev LOG_FORMAT=plain VAULT_ADDR="" pytest -q tests/ -v
ruff check app/

# Builds images
for svc in users-service products-service orders-service; do
    docker build -t "${svc}:1.0.0" -f "app/${svc}/Dockerfile" app/
done

# Charge / HPA
scripts/stress-hpa.sh --watch 90
```

Listing A.1. Aide-mémoire d'exploitation quotidienne

ANNEXE B

Variables d'environnement

Tableau B.1. Variables d'environnement

Variable	Défaut (.env.example)	Rôle
ENVIRONMENT	dev	active replis dev (sqlite mémoire, token éphémère)
SERVICE_NAME	users-service	construit le chemin Vault par service
LOG_LEVEL	DEBUG	verbosité logging
LOG_FORMAT	plain	json (prod) plain (dev)
VAULT_ADDR	http://localhost:8200	adresse API Vault
VAULT_TOKEN	(vide)	token direct (dev) ; alias VAULT_DEV_ ROOT_TOKEN_ID
DATABASE_URL	vide	secret par service
JWT_SECRET_KEY	vide	users-service
API_KEY	vide	products-service
PAYMENT_GATEWAY_ KEY	vide	orders-service

ANNEXE C

Index des captures demandées

Liste de contrôle des principaux fichiers attendus dans **images/** : les légendes des figures du corps du rapport font foi et précisent chacune le contenu attendu (161 emplacements au total, scénarios 1–7 compris) :

Tableau C.1. Index des captures demandées

Fichier	Contenu à capturer
architecture-globale.png	Schéma global VMs/cluster/namespaces/flux
reseau-libvirt.png	virsh net-dumpxml / vue réseau virt-manager
vms-topologie.png	virsh list + dominfo des 3 nœuds
ip-statique-dns.png	ip addr + ping inter-nœuds depuis master
exemple-emplacement.png	(capture d'exemple, remplaçable)
kvm-virsh-list.png	virsh list --all + dumpxml memory/vcpu
kvm-console-boot.png	Console VNC au boot Ubuntu 22.04
kvm-qcow2-info.png	qemu-img info base + volume nœud
cloudinit-userdata.txt.png	user-data rendu dans /var/lib/cloud
cloudinit-userdata.png	idem nommage principal
cloudinit-ssh-hardening.png	ssh root refusé vs devops accepté
cloudinit-fail2ban.png	fail2ban-client status sshd
terraform-init.png	init providers 0.9.8 / 2.9.0
terraform-plan.png	plan create N ressources
terraform-state-list.png	terraform state list
terraform-outputs.png	outputs master/worker ips
terraform-tfstate-gitignore.png	git status sans tfstate
terraform-inventory-gen.png	generate-inventory.sh + inventory.ini
ansible-playbook-start.png	list-tasks / début run
ansible-recap.png	PLAY RECAP vert
ansible-dpkg-hold.png	apt-mark showhold
ansible-nodes-ready.png	get nodes après run
ansible-join-perms.png	ls -l /tmp/kubeadm-join*
ansible-calico-running.png	Pods calico-system Running
docker-build.png	build multi-stage
docker-images-sizes.png	docker images/history tailles
docker-inspect-user-health.png	User appuser + Healthcheck
docker-run-healthy.png	docker ps healthy
containerd-crictl-ps.png	crictl ps sur un nœud

Fichier	Contenu à capturer
containerd-systemdcgroup.png	grep SystemdCgroup config.toml
containerd-status.png	systemctl status containerd
k8s-clusterinfo.png	cluster-info + version
k8s-controlplane-pods.png	pods kube-system sains
k8s-nodes-wide.png	get nodes -o wide IPs/containerd
k8s-calico-connectivity.png	ping/DNS inter-Pods inter-nœuds
kustomize-prod-render.png	kustomize build prod (replicas 3, digests)
k8s-limitrange-quota.png	describe limitrange + resourcequota
k8s-netpol-list.png	get networkpolicy (6)
k8s-netpol-deny-proof.png	flux refusé vs autorisé
k8s-hpa-status.png	get hpa targets/répliques
k8s-pdb.png	get pdb allowed disruptions
k8s-rolling-update.png	rollout watch maxSurge/maxUnavailable
k8s-securitycontext.png	describe pod security context
fastapi-swagger.png	/docs Swagger UI
fastapi-root.png	JSON racine vault_configured:true
fastapi-readyz-vault.png	readyz 200 vs 503
fastapi-failclosed-crash.png	CrashLoop + SystemExit logs
shared-logs-json.png	logs JSON event secret.fetch
shared-logs-plain-dev.png	format plain + warning default_used
shared-fail-closed-local.png	SystemExit local sans VAULT_ADDR
instrumentator-metrics.png	curl /metrics histogramme
instrumentator-grafana-app.png	dashboard perf alimenté
vault-status.png	pods + vault status unsealed
vault-mounts-auth.png	secrets list + auth list
vault-policy-read.png	policy read least-privilege
vault-kv-get.png	kv get users-service clés
vault-token-rotation.png	bootstrap rotation one-shot
vault-readyz-after-rotation.png	readyz 200 post-rotation
ci-actions-list.png	runs récents statuts/durées
ci-graph.png	graphe jobs du run
ci-lint-job.png	étapes lint vertes
ci-test-pipaudit.png	pip-audit x4 + pytest
ci-build-matrix.png	matrice 3 builds tags
ci-trivy-pass.png	table vide + SARIF upload
ci-security-tab.png	Code scanning alerts
ci-ghcr-packages.png	packages tags branche/sha/latest
ci-deploy-manual.png	dispatch production + deploy complet
gitleaks-clean.png	detect exit 0
gitleaks-detection.png	faux token détecté (contrôlé)
gitleaks-ci-job.png	job CI vert
trivy-scan-local.png	tableau CVE image
trivy-blocked.png	-exit-code 1 code retour 1
trivy-sbom.png	artefact SPDX aperçu
pipaudit-clean.png	audit strict clean
pipaudit-detection.png	CVE Flask simulées
precommit-allfiles.png	11 hooks Passed
precommit-fix.png	hook corrigeant + re-stage

Fichier	Contenu à capturer
conftest-pass.png	conftest sur manifests conformes
conftest-deny.png	3 deny messages contrôlés
kyverno-policies.png	get clusterpolicy Audit
ruff-clean.png	ruff check All checks passed
kubeconform-valid.png	find+kubeconform reproduisant CI
argocd-apps-grid.png	tuiles 12 Applications Synced
argocd-app-detail.png	arbre ressources + digest + historique
argocd-selfheal.png	scale manuel corrigé auto
argocd-prune.png	ressource supprimée par prune
argocd-rollback.png	rollback UI diff
prom-operator-cr.png	get prometheus READY
prom-targets-up.png	Targets tous UP
prom-promql-rps.png	graph rate(http_requests_total)
prom-servicemonitors.png	get servicemonitor liste
ksm-metrics.png	kube_hpa séries via port-forward
kubelet-scrape-targets.png	targets cadvisor 3 IP
am-ui-groups.png	alertes groupées/silences
am-rule-in-cluster.png	prometheusrule sync
am-hpa-firing.png	HPAPinnedAtMaxReplicas firing
am-slo-breach.png	5xx seuil + alerte
grafana-infra-overview.png	dashboard Infrastructure Overview
grafana-app-perf.png	RPS + p95/p99 en charge
grafana-error-rate.png	seuil 1 % visible
grafana-infra-detail-oom.png	ratio rouge + OOMKill
grafana-dash-configmaps.png	4 ConfigMaps labellisés
elk-pods-running.png	ES/filebeat/logstash/kibana Running
kibana-discover.png	index devops-platform-* logs JSON
kibana-search-vault.png	filtre event secret.fetch / 503
logstash-drop-proof.png	absence documents livez/readyz
es-cat-indices.png	indices quotidiens docs counts
slo-grafana-threshold.png	seuil SLO tracé
slo-recording-query.png	availability_30d ratio
validate-platform-all.png	7 contrôles + bonus verts
validate-platform-fail.png	-ci exit 1 contrôlé
validate-selfheal.gif	delete pod + retour <30 s
validate-security-ok.png	4 contrôles + TTL token
smoke-e2e.png	smoke-test complet vert
hpa-scaling-live.png	watch hpa/pods 2→5
ab-results.png	RPS + percentiles ab
grafana-under-load.png	dashboards pendant charge

Fin du rapport.

ANNEXE D

Arborescence commentée du dépôt

```
devops-central-platform/
|-- .github/workflows/ci-cd.yml # pipeline complet (447 lignes)
|-- .gitleaks.toml # config scan secrets (allowlist serree)
|-- .pre-commit-config.yaml # 11 hooks / 5 depots
|-- .dockerignore # contexte build minimal
|-- .env.example # variables documentees
|
|-- terraform/
| |-- main.tf # reseau, volumes, VMs, inventaire
| |-- variables.tf # 12 variables validees
| |-- outputs.tf # ips sensibles, inventaire
| |-- backend.tf # local actif; contrat S3+DynamoDB commente
| |-- cloud-init.tpl # durcissement boot
| '-- inventory.tpl # rendu [masters]/[workers]
|
|-- ansible/
| |-- ansible.cfg # forks=10, pipelining
| |-- playbook.yml # 5 plays (dont reset double opt-in)
| |-- requirements.yml # community.general==8.0.0
| |-- group_vars/all.yml # k8s 1.28, calico v3.26.1, CIDRs
| '-- roles/{docker,k8s_common,k8s_master,k8s_worker,k8s_reset}/
|
|-- app/
| |-- users-service/ # main.py + Dockerfile + requirements.txt
| |-- products-service/
| |-- orders-service/
| '-- shared/ # vault_client.py, log_config.py,
| # config.py, pyproject.toml
|
|-- k8s/
| |-- apps/ # base/ + users/products/orders + overlays
| |-- monitoring/
| | |-- base/ # ns monitoring + quota + limitrange
| | |-- prometheus/ # operator 0.74 + prom v2.51 (+helm alt.)
| | |-- alertmanager/ # am v0.27 + rules.yaml (SLO)
| | |-- grafana/ # 11.1 + sidecar + 4 dashboards CM
| | |-- elk/ # es/filebeat/logstash/kibana 8.14
| | |-- kubelet/ # scrape cadvisor explicite
| | '-- kube-state-metrics/ # v2.10.1
```

```
| |-- argocd/{install/,applications/} # v3.5.1 + AppProject + 12 Apps
| |-- vault/ # manifests dev-mode + policy.hcl
| '-- policies/ # conftest rego + kyverno audit
|
|-- scripts/
| |-- validate-platform.sh # 7 controles + self-heal
| |-- validate-security.sh # 4 controles securite
| |-- generate-inventory.sh # terraform -> ansible
| |-- bootstrap-vault-secret.sh # rotation stdin
| |-- bootstrap-elasticsearch-secret.sh
| |-- smoke-test.sh / stress-hpa.sh / stress-panel.py
|
'-- tests/test_services.py # pytest (ENVIRONMENT=dev)
```

Listing D.1. Structure complete du projet (fichiers cles)

ANNEXE E

Glossaire

ASGI Interface serveur-application asynchrone de Python (successeur de WSGI) : Uvicorn l'implémente, FastAPI s'y appuie.

ArgoCD Contrôleur GitOps : synchronise en continu le cluster avec les manifests d'un dépôt Git.

CIDR Notation d'un bloc d'adresses IP (ex. 192.168.0.0/16).

CNI Container Network Interface : plugin réseau des Pods ; Calico ici.

CRI Container Runtime Interface : API entre kubelet et runtime ; containerd ici.

DaemonSet Objet K8s exécutant un Pod sur chaque nœud (Filebeat, calico-node).

DevSecOps Intégration de la sécurité dans chaque étape du cycle DevOps.

Digest Empreinte immuable sha256 d'une image OCI.

fail-closed Politique refusant tout fonctionnement dégradé silencieux : sans secret résolvable, on refuse de démarrer.

GitOps Modèle opérationnel où Git est la source de vérité de l'état du système ; le cluster tire et réconcilie.

HEALTHCHECK Instruction Docker exécutant périodiquement un test de santé embarqué à l'image.

Helm/Kustomize Deux approches du packaging Kubernetes : templates paramétrables vs patches sur YAML concret.

HPA Horizontal Pod Autoscaler : ajuste le nombre de répliques selon CPU/mémoire.

IaC Infrastructure as Code : infrastructure décrite en code versionné (Terraform).

KV-v2 Moteur de secrets Vault avec historique de versions et soft-delete.

liveness/readiness/startup Les trois probes Kubernetes : vivant / prêt à servir / grâce au démarrage.

MTTD/MTTR Mean Time To Detect / To Recover : délais de détection et de rétablissement.

NetworkPolicy Pare-feu L3/L4 déclaratif entre Pods (Calico l'applique).

OOMKill Destruction d'un conteneur par le kernel lorsque sa limite mémoire est atteinte.

PDB PodDisruptionBudget : garantit un minimum de réplicas pendant les perturbations volontaires.

PromQL Langage de requête des séries temporelles Prometheus.

PSA restricted Profil Pod Security Admission le plus strict : pas de root, caps limitées, seccomp obligatoire...

Recording rule Règle Prometheus pré-calculant une expression coûteuse en une nouvelle série.

SBOM Software Bill of Materials : inventaire normalisé des composants d'une image (SPDX ici).

ServiceMonitor CRD du Prometheus Operator déclarant une cible de scraping.

SLO/SLI Service Level Objective / Indicator : objectifs mesurés par indicateurs (99,9 %, p95...).

sync waves Ordonnancement ArgoCD : les ressources de wave N attendent la santé des waves précédentes.

taint/toleration, affinity Mécanismes de placement des Pods sur les nœuds.

ANNEXE F

Journal des décisions (extraits ADR)

Format court : Décision → Contexte → Conséquence.

Tableau F.1. Journal des décisions (extraits ADR)

Décision	Contexte	Conséquence
Terraform épinglé ~1.5	Éviter ruptures 1.6+	CI rejoue exactement 1.5.7 ; migration OpenTofu différée
4096 Mo par nœud	kubelet NodeStatusUnknown à 2 Go sous ELK+ArgoCD	Homelab plus gourmand mais stable
Inventaire généré par Terraform	Inventaires manuels divergent toujours	zéro édition main ; -no-refresh disponible
Addons kubeadm différés	CoreDNS CrashLoop sans CNI	init en deux phases dans Ansible
Calico fail-closed	Un CNI cassé = cluster mort	attente Ready bloquante, abort assumé
dpkg hold kubelet, kubeadm, kubectl	apt upgrade casse le pin 1.28	holds posés par Ansible
no_log sur join command	token bootstrap dans logs CI	fichiers 0600 sur master
Secrets applicatifs fail-closed	JWT signable avec clé dev = fuite	SystemExit au démarrage prod
Deploy manuel uniquement	Prod jamais poussée automatiquement	workflow_dispatch + environnement GitHub
Tags images branche+sha	:latest mutable = rollback impossible	latest réservé branche par défaut
Trivy bloquant CRITICAL/HIGH unfixed	Pipeline vivant malgré faux positifs	SARIF pour le suivi, SBOM archivé

Décision	Contexte	Conséquence
conftest/Kyverno Audit	en Mesurer le bruit avant bloquer	passage enforce prévu après stabilisation
Vault dev mode assumé	Lab homelab, TLS/raft lourds	chemin production documenté, injector prêt
GitOps pull (ArgoCD)	CI push = credentials cluster côté SaaS	selfHeal + prune + revert = rollback

ANNEXE G

Cartographie des ports et endpoints

Tableau G.1. Cartographie des ports et endpoints

Composant	Port	Usage
Microservices (3×)	8000	HTTP app + /metrics + probes
Services K8s apps	80	ClusterIP vers 8000
Vault	8200	API KV/auth
Vault metrics	8201	/v1/sys/metrics
Prometheus	9090	UI + TSDB
AlertManager	9093	UI + API alertes
Grafana	3000	UI dashboards
Elasticsearch	9200 / 9300	HTTP / transport
Logstash beats	5044	entrée Beats (réserve)
Logstash monitor	9600	API node stats
Kibana	5601	UI exploration
kubelet metrics	10250	https-metrics/cadvisor
SSH nœuds	22	clés uniquement (durci)
Endpoints HTTP app	—	/ · /livez · /readyz · /metrics · /users · /products · /orders

ANNEXE H

Checklists d'exploitation

H.1 Quotidienne (5 min)

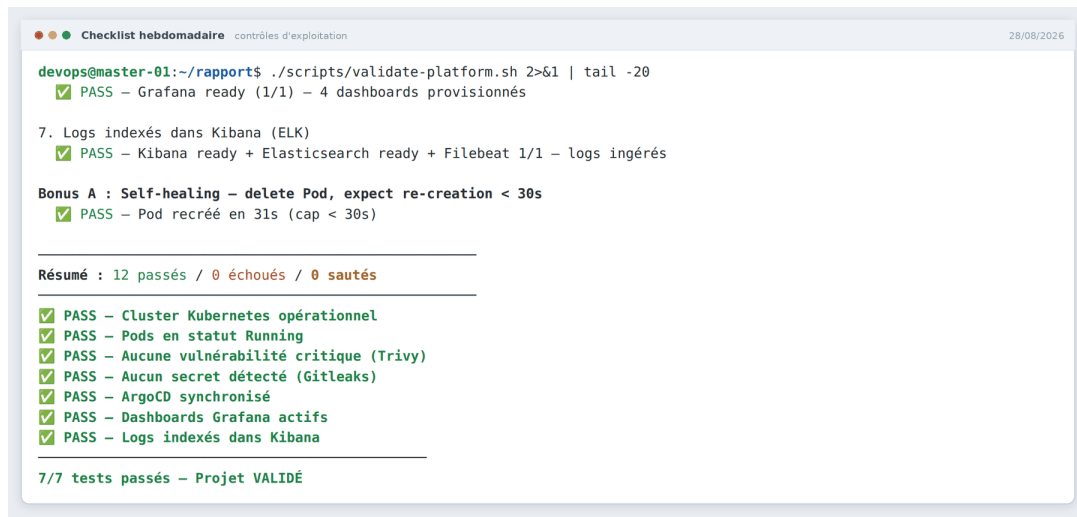
1. run nocturne CI vert ? (sinon : CVE/drift → Annexe runbooks)
2. `kubectl get pods -A | grep -v Running` : vide ?
3. UI ArgoCD : toutes Applications Synced/Healthy ?
4. Grafana Error Rate SLO <1 % et p95 <200 ms sur 24 h ?
5. AlertManager : aucun firing non acquitté ?

H.2 Hebdomadaire (20 min)

1. `scripts/validate-platform.sh` complet avec bonus self-heal ;
2. `scripts/validate-security.sh` : TTL token Vault vérifié ;
3. Prometheus targets : zéro down depuis 7 jours ;
4. Elasticsearch : taille indices, rétention 15 j respectée ;
5. GHCR : purger les tags de branches supprimées ;
6. revue des silences AlertManager créés (à lever si obsolètes).

H.3 Mensuelle (1 h)

1. rotation du token root Vault + preuve `validate-security` ;
2. rotation `JWT_SECRET_KEY/API_KEY/PAYMENT_GATEWAY_KEY` ;
3. mise à jour mineure des dépendances (PR bump + chaîne complète) ;
4. test restore : recréation VM via Terraform `destroy/create` sur worker de test ;
5. revue des quotas `ResourceQuota` vs consommation réelle ;
6. relecture du tableau menaces/contre-mesures (chaque ligne encore vraie ?).



```
devops@master-01:~/rapport$ ./scripts/validate-platform.sh 2>&1 | tail -20
✅ PASS - Grafana ready (1/1) - 4 dashboards provisionnés

7. Logs indexés dans Kibana (ELK)
✅ PASS - Kibana ready + Elasticsearch ready + Filebeat 1/1 - logs ingérés

Bonus A : Self-healing - delete Pod, expect re-creation < 30s
✅ PASS - Pod recréé en 31s (cap < 30s)

-----
Résumé : 12 passés / 0 échoués / 0 sautés
-----
✅ PASS - Cluster Kubernetes opérationnel
✅ PASS - Pods en statut Running
✅ PASS - Aucune vulnérabilité critique (Trivy)
✅ PASS - Aucun secret détecté (GitLeaks)
✅ PASS - ArgoCD synchronisé
✅ PASS - Dashboards Grafana actifs
✅ PASS - Logs indexés dans Kibana

7/7 tests passés - Projet VALIDÉ
```

Figure H.1. Checklist hebdo exécutée en terminal : validate-platform 7+bonus verts.

ANNEXE I

Banque de questions / réponses éclair

Alignée sur l'objectif du projet (raconter les choix au format STAR en entretien).

Tableau I.1. Banque de questions / réponses éclair

Question	Réponse éclair
Pourquoi Terraform plutôt que Ansible pour créer les VMs ?	Gestion d'état + plan/prévisualisation ; Ansible est idempotent mais sans état : la dérive serait invisible. Les deux se complètent (provision vs configure).
Pourquoi containerd et pas Docker runtime ?	Depuis 1.24, dockershim supprimé ; containerd = CRI natif, surface réduite, chemin industriel. Docker reste pour le build.
SystemdCgroup, pourquoi c'est vital ?	kubelet et runtime doivent partager le même driver cgroup sinon OOMKills fantômes et évictions injustifiées.
Pourquoi différer CoreDNS/kube-proxy après Calico ?	Sans CNI, ces addons CrashLoop ; init en deux phases = bootstrap déterministe.
Pourquoi Calico vs Flannel ?	NetworkPolicy ingress+egress complète : le modèle default-deny du projet en dépend.
Kustomize ou Helm pour vos apps ?	Kustomize : patch sans template, diffs Git lisibles, natif ArgoCD. Helm gardé pour stacks tierces paramétrables.
readiness vs liveness ici ?	livez = processus vivant (aucune dépendance) ; readyz = Vault joignable. Séparation = pas de crash inutile, mais sortie propre du Service.
Que se passe-t-il si Vault tombe ?	readyz 503 → endpoints vidés → zéro trafic servi dégradé. livez OK donc pas de redémarrage inutile. Rétablissement = retour automatique.

Question	Réponse éclair
Fail-closed, pourquoi refuser de démarrer ?	Un fallback silencieux signerait des JWT avec une clé prévisible : pire que l'indisponibilité.
Pourquoi un token root hors bande ?	Jamais en argv (ps), disque (mktemp 0600), ni Git (gitleaks) ; stdin only, affiché une fois.
Comment garantissez-vous zéro secret commité ?	Double gitleaks (pre-commit + CI historique full depth), allowlist resserrée, tfstate gitignored + exclu scan.
Trivy bloque sur quoi exactement ?	CRITICAL,HIGH exit-code 1 ignore-unfixed os,library : bloquant seulement ce qui est corrigeable.
pip-audit -strict, l'intérêt du strict ?	Toute erreur d'audit devient un échec : jamais de succès silencieux trompeur.
Pourquoi deploy manuel uniquement ?	La prod ne doit pas être poussée automatiquement ; workflow_dispatch + environnement GitHub = porte humaine.
Pourquoi tags branche+sha, pas latest ?	Immutabilité : chaque déploiement retraçable au commit ; rollback fiable. latest réservé à la branche par défaut.
GitOps vs CI push classique ?	Le cluster tire depuis Git : selfHeal annule toute dérive, prune purge les orphelins, rollback = git revert, audit = git log.
Sync waves ArgoCD, exemple ?	-100 argocd-install → 5 base monitoring → 15 prometheus → 20 alertmanager/elk/rules → 25 grafana. Ordre garanti par santé.
SLO 99,9%/30 j, comment calculé ?	Recording rule avg_over_time(up[30d]) par job, alerte si $(1 - ratio) \times 100 > 0,1$ pendant 10 min.
MTTD <2 min, comment tenu ?	scrape 15 s + for 2-10 min + groupement AM 30 s.
HPA anti-flapping ?	Montée +100%/30 s (réactivité) ; descente stabilisation 300 s puis -25%/min (prudence).
Alerte HPA pinned, pourquoi ?	HPA collé au max 15 min = fuite probable qui scale indéfiniment : incident réel n°9.
OOMKill, quelle chaîne ?	LimitRange défauts → limites pod → ratio 80 % alerte warning → kernel kill → alerte critical + auto-réparation.
NetworkPolicy default-deny, coût ?	Six politiques déclaratives ; tout flux non listé est refusé au noyau par Calico. Testable par pod jetable.

Question	Réponse éclair
PSA restricted, effet concret ?	Aucun Pod root, caps non droppées interdits dans devops-platform : contrainte d'admission, pas une convention.
conftest vs Kyverno ?	Même politique à deux instants : conftest en CI (avant push cluster), Kyverno à l'admission (dernier rempart). Audit aujourd'hui, enforce demain.
kubeconform vs kubectl apply dry-run ?	Hors ligne, rapide, schémas 1.28 épinglés, intégré preflight du deploy.
Pourquoi dashboards-as-code ?	ConfigMaps labellisés + sidecar : reviewés en PR, versionnés, restaurables : comme tout manifest.
ELK dimensionné comment ?	Heaps JVM 512/256 Mo, queues bornées, PVC 10 Gi : calibré VM 4 Go ; contrainte réelle apprise tôt.
Filebeat direct ES ou Logstash ?	Config active : direct ES. Logstash déployé prêt à s'intercaler (enrichissement) quand nécessaire.
k6 absent, conséquence ?	Job nocturne load-test rouge : lacune connue n°1 ; ab couvre l'urgence interactive en attendant.

ANNEXE J

Matrice outils × dimensions DevOps

Lecture : pour chaque dimension de la plateforme, les outils qui la portent.

Tableau J.1. Matrice outils dimensions DevOps

Dimension	Outils impliqués (rôle spécifique)
Provisionner	libvirt/KVM (VMs) · cloud-init (boot) · Terraform (déclaration + état) · provider libvirt 0.9.8 · local 2.9.0
Configurer	Ansible (6 rôles) · community.general (modprobe) · dpkg hold · sysctls/modules
Conteneuriser	Docker CE (builds) · BuildKit 1.6 · python:3.11-slim · .dockerignore
Orchestrer	kubeadm · Kubernetes 1.28 · containerd · Calico v3.26.1 · Kustomize 5.4.3 · HPA/PDB/topology
Sécuriser le code	Gitleaks 8.18 (local+CI) · Ruff 0.1.9 · yamllint 1.33 · pre-commit (11 hooks)
Sécuriser les images	Trivy v0.24 (gate+SARIF+SBOM) · multi-stage non-root · HEALTHCHECK · labels OCI
Sécuriser l'exécution	PSA restricted · LimitRange · ResourceQuota · NetworkPolicies ×6 · securityContext strict · conftest Rego ×3 · Kyverno ×2
Gérer les secrets	Vault 1.15.2 · KV-v2 · auth Kubernetes TTL 1h · hvac 2.1.0 · bootstrap stdin · fail-closed applicatif
Livrer en continu	GitHub Actions (7 jobs, cron 03h00) · GHCR · metadata tags immuables · provenance/SBOM attestations · kustomize digest pinning
Réconcilier	ArgoCD v3.5.1 · AppProject · 12 Applications · sync waves · prune/selfHeal · ServerSideApply
Observer les métriques	Prometheus Operator 0.74/v2.51 · ServiceMonitors 15 s · kubelet cadvisor · kube-state-metrics 2.10.1 · Instrumentator 6.1

Dimension	Outils impliqués (rôle spécifique)
Alerter	AlertManager v0.27 · PrometheusRule SLO ×3 + incident ×2 + recording rule
Visualiser	Grafana 11.1 · sidecar 1.27.6 · 2 datasources · 4 dashboards provisionnés
Centraliser les logs	Filebeat 8.14 DS autodiscovery · Elasticsearch 8.14 mono-nœud durci · Logstash pipeline · Kibana
Valider de bout en bout	validate-platform.sh (7+bonus) · validate-security.sh (4) · smoke-test.sh · stress-hpa.sh (ab) · k6 (à écrire) · pytest
Documenter/décider	ADR inline dans le code · post-mortems incident=« N » · ce rapport

ANNEXE K

Feuille de route de durcissement

K.1 Vault : du dev-mode à la production

1. storage raft (3 rélicas) + TLS interne (autounseal KMS optionnel) ;
2. bascule des Pods vers l'Agent Injector (annotations + SA déjà liés aux rôles d'auth) : suppression du token root partagé ;
3. politiques par service déjà prêtes (read-only) ; ajouter response wrapping pour les tokens courts ;
4. audit devices activés (logs Vault vers ELK).

K.2 Politiques : Audit → Enforce

1. mesurer 2 semaines de conftest/Kyverno en Audit (faux positifs réels) ;
2. corriger les exceptions légitimes dans les manifests ;
3. basculer Kyverno validationFailureAction: Enforce, background: true ;
4. ajouter règles : labels obligatoires, resources requests/limits requis.

K.3 CI/CD

1. écrire tests/k6/load-test.js (priorité n°1) ;
2. corriger deploy : outputs par matrice (job outputs map) pour digests distincts ;
3. trivy-scan : épingler sur le digest du build plutôt que le tag ;
4. OIDC AWS pour le backend Terraform S3+DynamoDB (contrat déjà écrit).

K.4 Application

1. PostgreSQL réel (NetworkPolicy 5432 déjà réservée) ;

2. JWT effectif (clé déjà chargée) : middleware sign/verify ;
3. overlay dev : retirer les HPA au lieu de patcher les réplicas.

ANNEXE L

Correspondance alertes / incidents / runbooks

Tableau L.1. Correspondance alertes / incidents / runbooks

Alerte	Étiquette	Réflexe immédiat
ContainerMemoryRatio High	incident=1	identifier le pod ; vérifier limites vs usage réel ; scaler horizontal si charge légitime
PodEvictionOngoing	incident=1	describe (OOMKilled ?), remonter la limite via overlay, laisser l'auto-réparation recréer
HPAPinnedAtMax-Replicas	incident=9	chercher une fuite mémoire ; sinon revoir maxReplicas ou la charge
SLOAvailabilityBreach	slo=availability	up des jobs ; ArgoCD santé ; incidents réseau/CNI
SLOLatencyP95Breach	slo=latency	handler fautif via p95 par endpoint ; logs lents ; saturation CPU
SLO5xxErrorRateBreach	slo=error-rate	dashboard Error Rate → topk handler → logs filtrés status_code

ANNEXE M

Guide de réalisation des captures

M.1 Conventions générales

- format PNG (ou GIF pour le self-heal), résolution native, fond lisible ;
- terminal : police $\geq 14\text{pt}$, thème clair, prompt visible, commande ET sortie dans la même capture ;
- UI web (Grafana/Kibana/ArgoCD/Swagger) : plein écran, panneau pertinent ouvert, horloge visible quand le temps compte ;
- masquer toute valeur sensible (tokens, mots de passe) : les scripts du projet affichent déjà `-redact` / `one-shot` ;
- nommer le fichier exactement comme indiqué dans l'emplacement réservé puis le déposer sous `images/` : la figure est intégrée automatiquement à la compilation suivante.

M.2 Ordre conseillé de prise de captures

1. plateforme up + validate-platform vert (base saine pour toutes les autres) ;
2. captures statiques infra/cluster/kustomize ;
3. interfaces web (ArgoCD, Grafana, Kibana, Swagger, Prometheus) ;
4. CI/GitHub Actions (un run complet frais) ;
5. scénarios 2–7 dans l'ordre (chaque scénario se termine par un état rétabli) ;
6. en dernier : captures destructives ou « rouges » (fail-closed, trivy bloqué, run nocturne rouge).

```
devops@master-01:~/rapport$ kubectl top nodes; echo ;
kubectl get pods -A --no-headers | awk '{print $1}' | sort | uniq -c | sort -rn
NAME                                CPU(cores)   CPU(%)       MEMORY(bytes) MEMORY(%)
devops-platform-control-plane      537m         4%           10450Mi      32%

10 monitoring
10 devops-platform
9 kube-system
7 p-dae2622f77640c3b1af
7 argocd
6 p-2cb9b768730b4701a3f1
4 kyverno
3 tekton-pipelines
2 vault
2 p-e3fcd7d7d3b24422ad08
2 p-b83cbbc7f42240b4bace
2 p-a47ffc6bba304f9f83cd
2 p-8297044aee024471bc71
2 p-801b1e3300d449e79ce1
2 p-27d102eb25504676b085
2 p-125a57a274fc4f4299f0
2 p-0c84ff512da040f098d9
2 p-088270b4ea0f4a28b984
2 logging
2 gitops
1 tekton-pipelines-resolvers
1 p-f7ef268037eb4e6eb606
1 p-85e87bace0174d4db5a6
1 local-path-storage
1 ingress-nginx
```

Figure M.1. Capture « carte d'identité » de la plateforme : `kubectl top nodes` + `kubectl get pods -A` côte à côte.

M.3 Rappels utiles pendant la capture

```
kubectl port-forward svc/grafana -n monitoring 3000:3000 &
kubectl port-forward svc/prometheus -n monitoring 9090:9090 &
kubectl port-forward svc/kibana -n monitoring 5601:5601 &
kubectl port-forward svc/users-service -n devops-platform 18080:80 &
# ArgoCD :
kubectl port-forward svc/argocd-server -n argocd 8443:443
```

Listing M.1. Port-forwards typiques pour les UI